COS120 - CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

BÀI 6: CÂY VÀ CÂY NHỊ PHÂN TÌM KIẾM

MỤC TIÊU

- Hiểu được cấu trúc, các khái niệm và hoạt động của cây tổng quát, cây nhị phân, cây nhị phân tìm kiếm.
- Biết cách hiện thực các thao tác trên cây như chèn, xóa, tìm kiếm, duyệt cây,...
- Biết được các ứng dụng của cây trong thực tế và sử dụng cây trong lập trình để giải quyết các bài toán thực tế.

NỘI DUNG

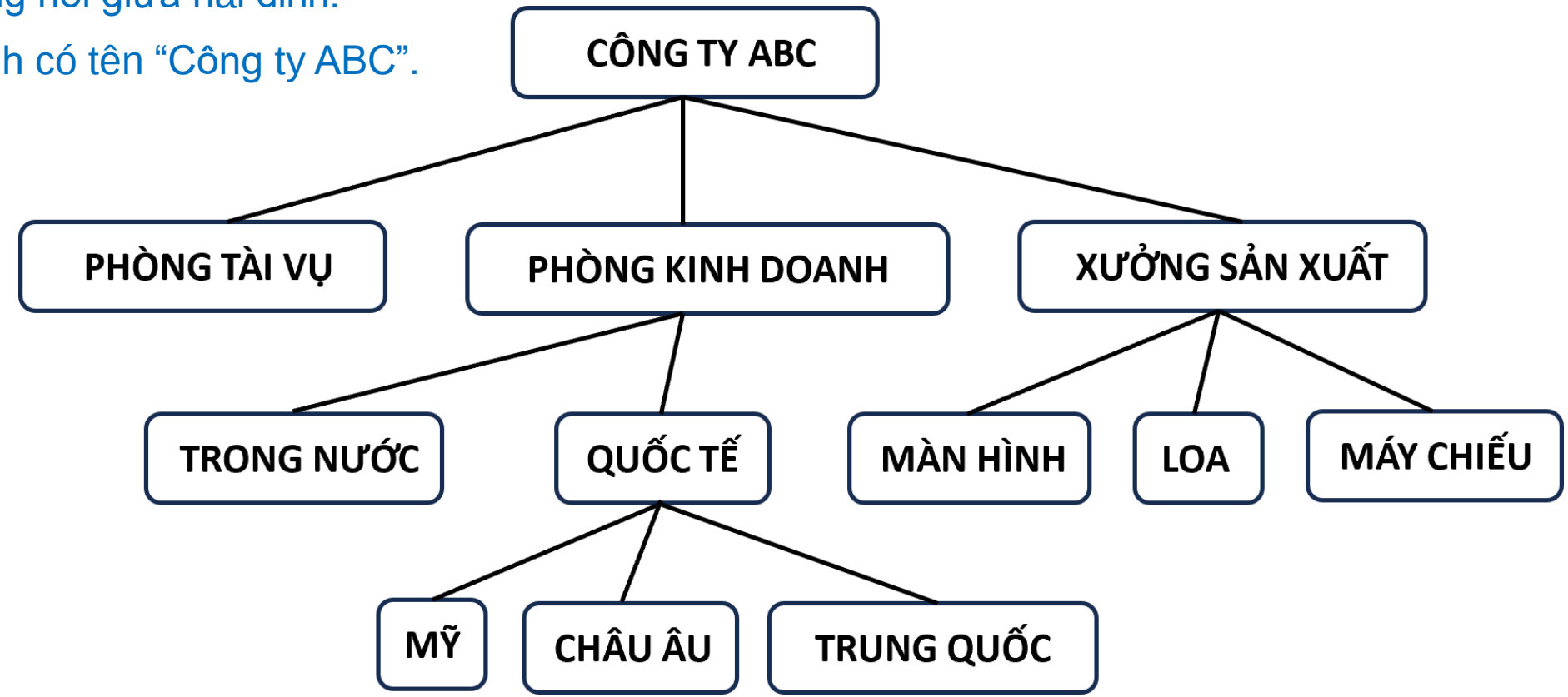
- 6.1 - CẤU TRÚC CÂY TỔNG QUÁT
- 6.2 - CÂY NHỊ PHÂN
- 6.3 - CÂY NHỊ PHÂN TÌM KIẾM - BST

CÂY VÀ CÂY NHỊ PHÂN TÌM KIẾM

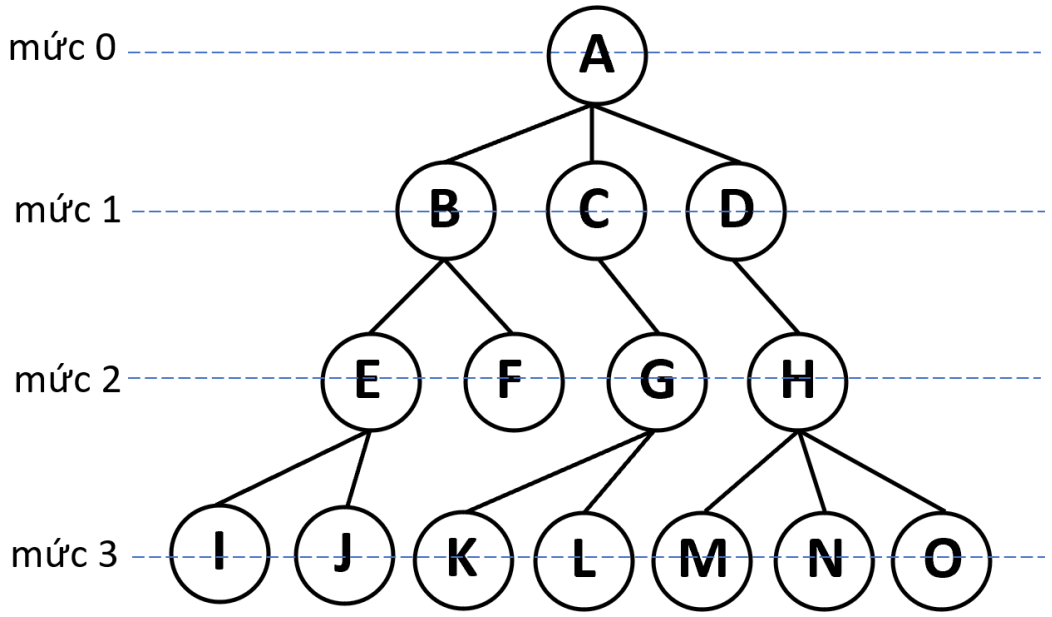
6.1 - CẤU TRÚC CÂY TỔNG QUÁT

KHÁI NIỆM CÂY TỔNG QUÁT

- Sơ đồ của một công ty được biểu diễn dạng cây.
- Các đỉnh là tên của các đơn vị (công ty, phòng, ban).
- Cạnh là đường nối giữa hai đỉnh.
- Nút gốc là đỉnh có tên “Công ty ABC”.

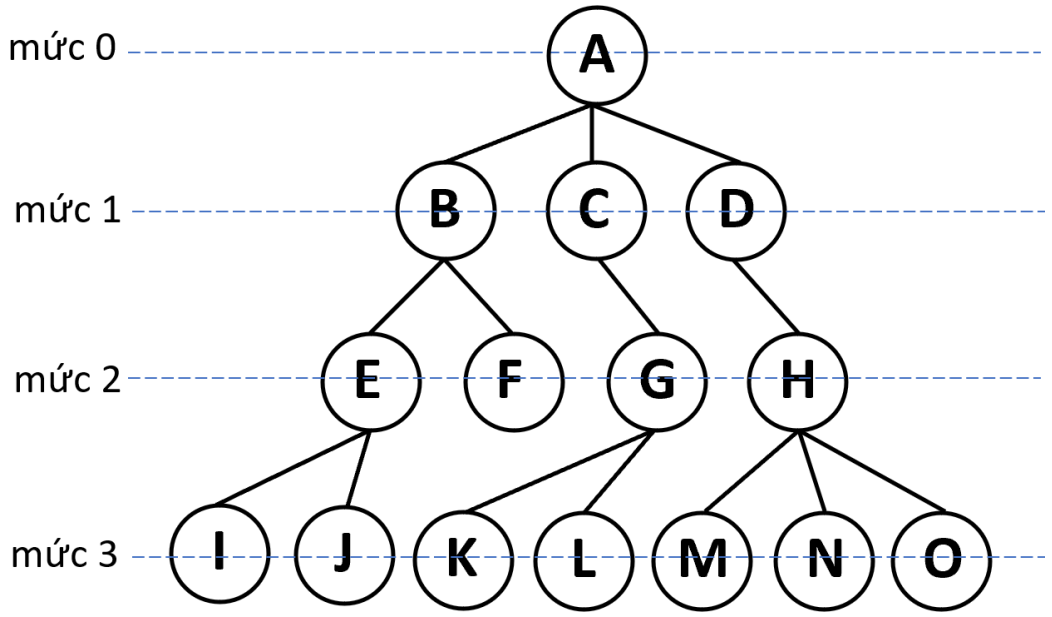


KHÁI NIỆM CƠ BẢN VỀ CÂY



- **Nút gốc (Root Node)**: nút đầu tiên của cây, không có nút cha. Trong hình nút A là nút gốc.
- **Nút lá (Leaf Node):** nút không có nút con. Nút F, I, J, K, L, M, N, O là các nút lá.
- **Nút trung gian (Internal Node):** nút có ít nhất một nút con. Không phải là nút gốc và cũng không phải là nút lá. Trong hình, các nút B, C, D, E, G, H là các nút trung gian.
- **Nút anh em (Sibling/Brother Nodes):** các nút có cùng một nút cha. {B, C, D} cha là A, {E, F} cha là B, {I,J} cha là E, {K, L} cha là G, {M, N, O} cha là H.
- **Bậc của nút (Degree of Node):** bằng với số lượng nút con trực tiếp của một nút. Bậc 3 (A,H). Bậc 2 (B, E, G). Bậc 1 (C, D). Bậc 0 (F, I, J, K, L, M, N, O).
- **Bậc của cây (Degree of Tree):** là bậc lớn nhất trong số các bậc của các nút trong cây. Cây trong hình có bậc **3**.

KHÁI NIỆM CƠ BẢN VỀ CÂY



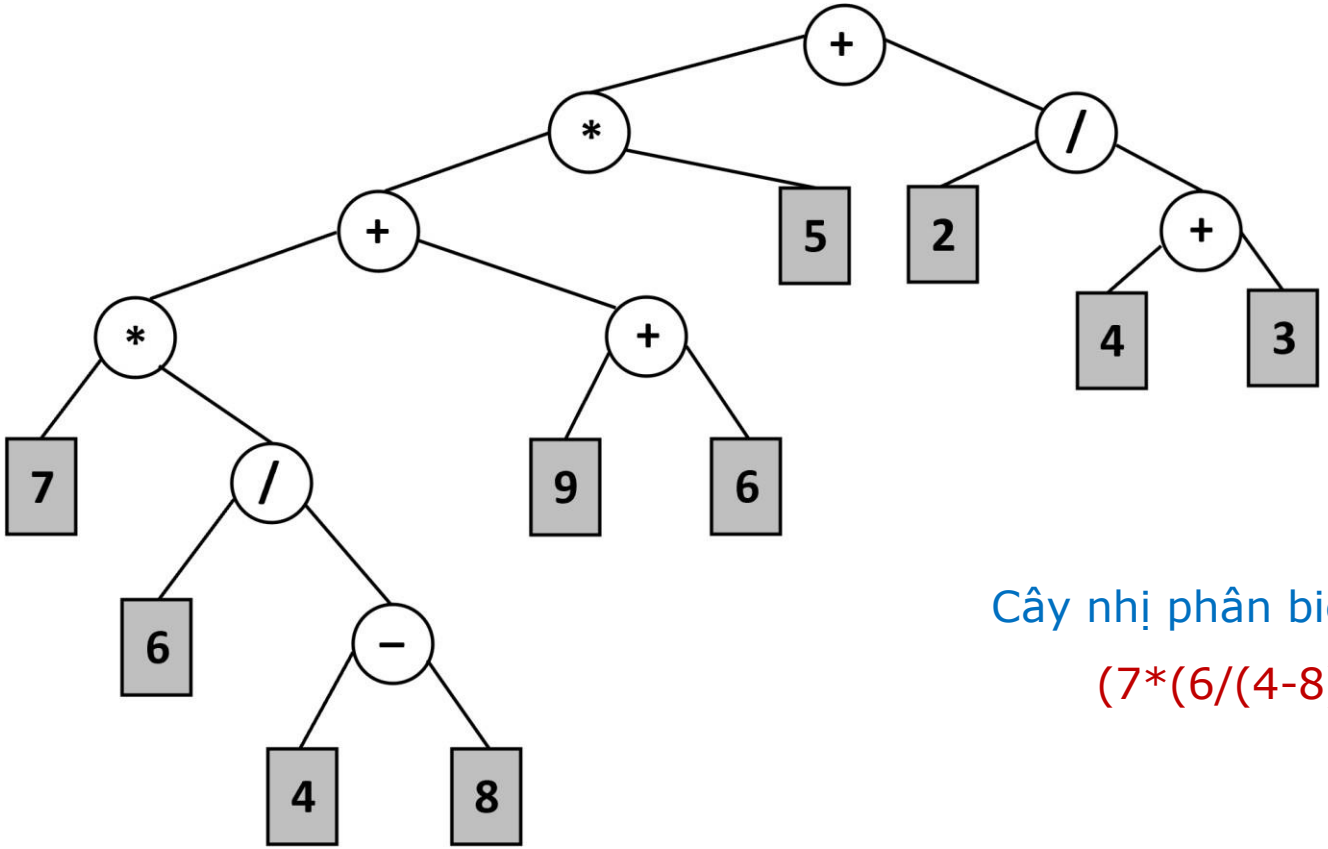
- **Mức của nút (Level of Node): Nút gốc, mức 0**. Mức của các nút khác**: lấy mức nút cha cộng 1**. A mức 0. Mức 1 (B, C, D). Mức 2 (E, F, G, H). Mức 3 (I, J, K, L, M, N, O).
- **Chiều cao của cây (Height of Tree):** bằng với số lượng cạnh từ nút **gốc** đến **nút lá xa nhất**. Chiều cao cây trong hình bằng 3.
- **Đường đi (Path):** ở đây, ta chỉ xét các đường đi xuất phát từ **nút gốc** đến một **nút bất kỳ trong** cây. Nó là một chuỗi các cạnh nối liền từ nút gốc đến nút cần đến (nút đích). Trong hình, đường đi từ gốc đến nút G là A→C→G. Đường đi từ A đến J là A→B→E→J.
- **Chiều dài đường đi (Path Length):** là **số lượng cạnh trên đường đi**. Trong hình, chiều dài đường đi từ A đến J là 3 (vì nó đi qua 3 cạnh: AB, BE, EJ).

CÂY VÀ CÂY NHỊ PHÂN TÌM KIẾM

6.2 - CÂY NHỊ PHÂN

KHÁI NIỆM CÂY NHỊ PHÂN

- Định nghĩa: cây **nhị phân** là cây mà **mỗi nút** có **tối đa hai** nút con.
- **Hai nút con** này thường được gọi là **nút con trái** và **nút con phải**.

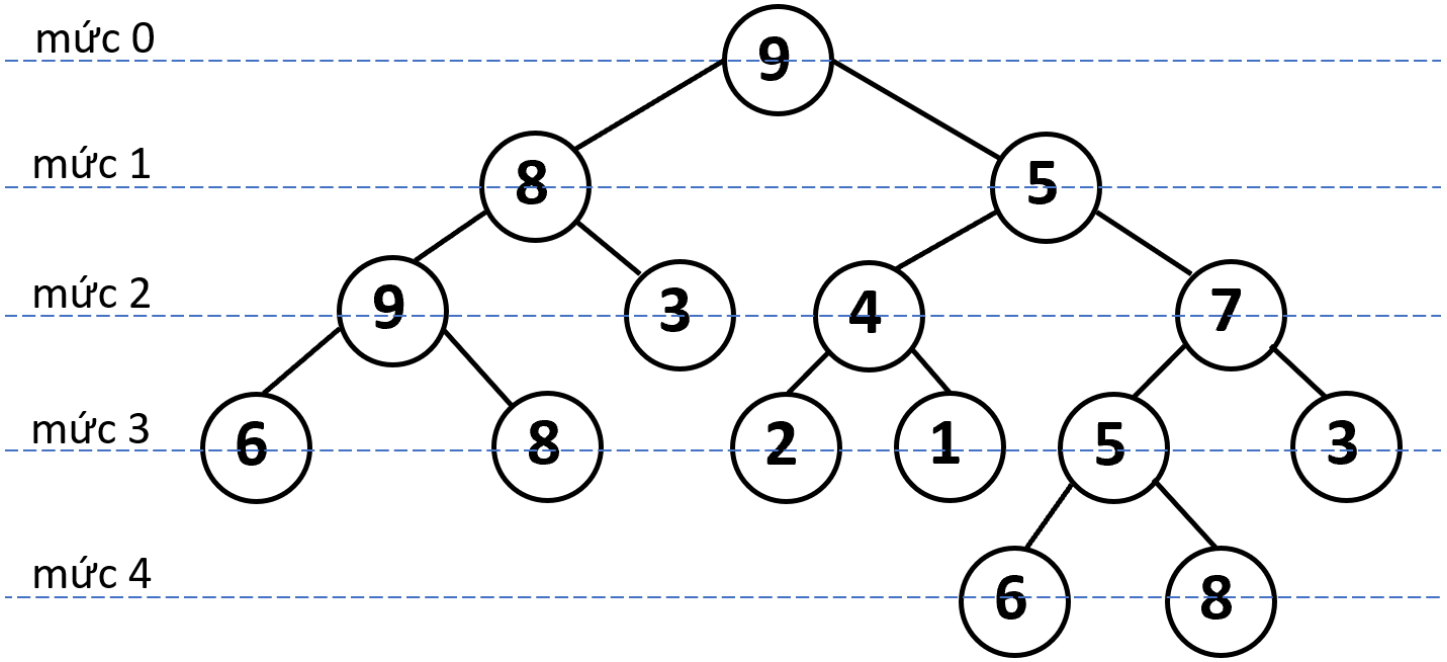


Cây nhị phân biểu diễn biểu thức toán học.

(7*(6/(4-8))+9+6)*5+2/(4+3)

MỘT SỐ TÍNH CHẤT CỦA CÂY NHỊ PHÂN

- Gọi n là lượng nút của cây, mức là m, chiều cao h.



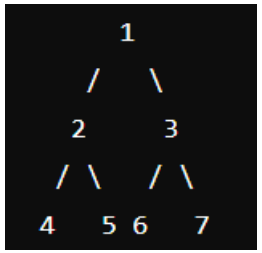
1) Số nút nằm ở mức m thì $\leq 2^m$
2) Số nút lá $\leq 2^{h-1}$
3) Chiều cao của cây $h \geq \log_2 n$
4) Số nút trong cây $n \leq 2^{h-1}$

CÁC CÂY NHỊ PHÂN ĐẶC BIỆT

- Cây nhị phân đầy đủ (Full Binary Tree)
- Cây nhị phân hoàn chỉnh (Complete Binary Tree)
- Cây nhị phân cân bằng (Balanced Binary Tree)
- Cây nhị phân tìm kiếm (Binary Search Tree – BST)
- Cây AVL (AVL Tree)- Cây nhị phân tìm kiến cân bằng
- Cây đỏ-đen (Red-Black Tree)
- Cây Heap (Heap Tree)

CÁC CÂY NHỊ PHÂN ĐẶC BIỆT – VÍ DỤ

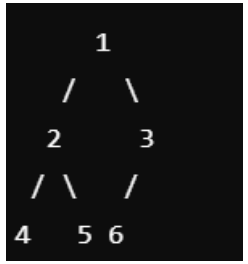
- Cây nhị phân đầy đủ (Full Binary Tree)



Điều kiện:

•Cây nhị phân đầy đủ là **cây** mà **mọi** nút hoặc **không** có con nào, hoặc có **đúng 2** con (tức là mọi nút đều có 0 hoặc 2 con).

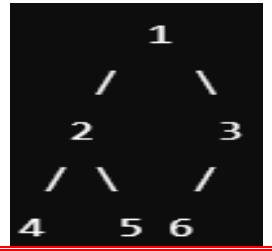
- Cây nhị phân hoàn chỉnh (Complete Binary Tree)



Điều kiện:

•Cây nhị phân hoàn chỉnh là **cây** mà **mọi mức đều được điền đầy đủ**, ngoại trừ mức cuối cùng, trong đó các nút phải được lấp đầy từ trái sang phải.

- Cây nhị phân cân bằng (Balanced Binary Tree)



Điều kiện:

•Cây nhị phân cân bằng là cây mà **sự chênh lệch** chiều cao giữa hai cây **con** của **mọi** nút không lớn hơn **1**.

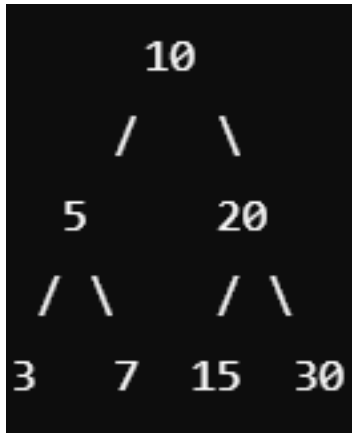
CÁC CÂY NHỊ PHÂN ĐẶC BIỆT – VÍ DỤ

- Cây nhị phân tìm kiếm (Binary Search Tree – BST)

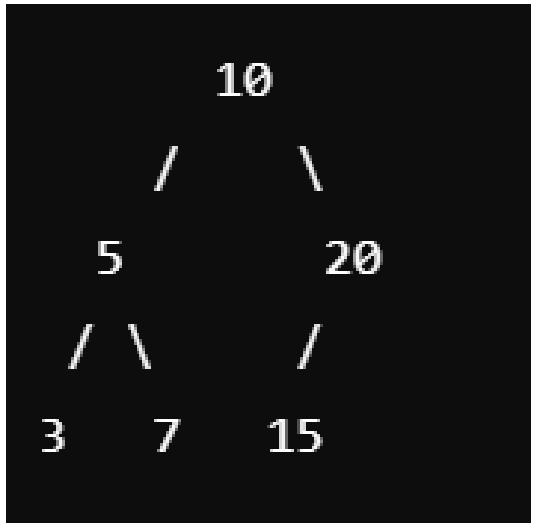
Điều kiện:

Cây nhị phân tìm kiếm (**BST**) là **một dạng cây nhị phân** thỏa mãn ĐK sau:

1. **Giá trị** của mỗi **nút con bên trái** nhỏ hơn giá trị của **nút cha**.
2. Giá trị của mỗi nút con bên phải **lớn hơn** giá trị của nút cha.
3. Các điều kiện này phải đúng **cho mọi nút trong cây**.



- Cây AVL (AVL Tree)



Điều kiện:

Cây AVL là một loại **cây nhị phân tìm kiếm** đặc biệt, được **cân bằng theo chiều cao**. Cây AVL thỏa mãn các điều kiện sau:

1. Là một **cây nhị phân tìm kiếm** (BST).
2. Đối với **mỗi nút** trong cây, sự **chênh lệch chiều cao** giữa hai cây con của nó (**còn gọi là hệ số cân bằng**) **không** được **lớn hơn 1**. **Hệ số cân bằng** của **một nút** là: **Balance Factor=Height of left subtree−Height of right subtree**
3. Nếu có **sự mất cân bằng** xảy ra sau các thao tác như **chèn hoặc xóa**, cây sẽ tự động được cân bằng lại **bằng các phép xoay (rotation)**.

CÁC CÂY NHỊ PHÂN ĐẶC BIỆT – VÍ DỤ

❑Cây đỏ-đen (Red-Black Tree)

```
      10(B)
     /    \
  5(R)     20(B)
 /   \        \
3(B)  7(B)    30(R)
```

Điều kiện:

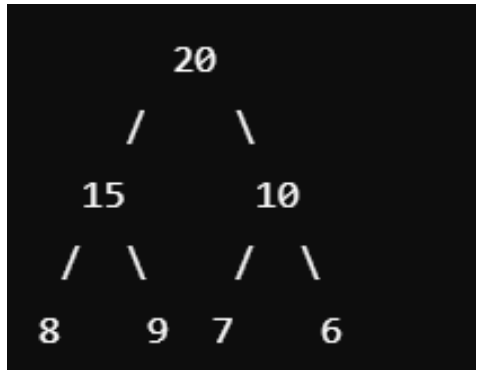
Cây đỏ-đen là **một loại cây nhị phân tìm kiếm** (BST) có các đặc tính bổ sung giúp duy trì cân bằng cao, **đảm bảo các thao tác như tìm kiếm, chèn và xóa** đều có thời gian trung bình là **O(logn)**. Mỗi nút trong cây đỏ-đen tuân theo các quy tắc sau:

1. **Mỗi nút** hoặc là đỏ hoặc là đen.
2. **Gốc** của cây luôn là **màu đen**.
3. **Không** có hai nút đỏ liên tiếp trên một đường đi từ gốc đến lá (tức là một nút đỏ không thể có **con** màu đỏ).
4. **Mọi đường đi từ một nút** đến các **lá con** đều có **cùng số lượng nút đen**.
5. **Các nút lá** đều là nút giả (null) và **được coi là nút đen**.

CÁC CÂY NHỊ PHÂN ĐẶC BIỆT – VÍ DỤ

- Cây Heap (Heap Tree): Cây Heap là một cây nhị phân hoàn chỉnh thỏa mãn tính chất Heap, có hai loại:

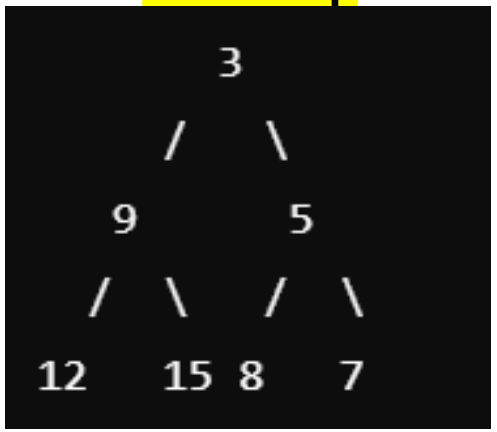
1. **Max-Heap**



Điều kiện:

- **Mọi nút đều có giá trị lớn hơn hoặc bằng** giá trị của **các con**.
- **Gốc của cây** sẽ chứa giá trị **lớn nhất**.

2. **Min-Heap**



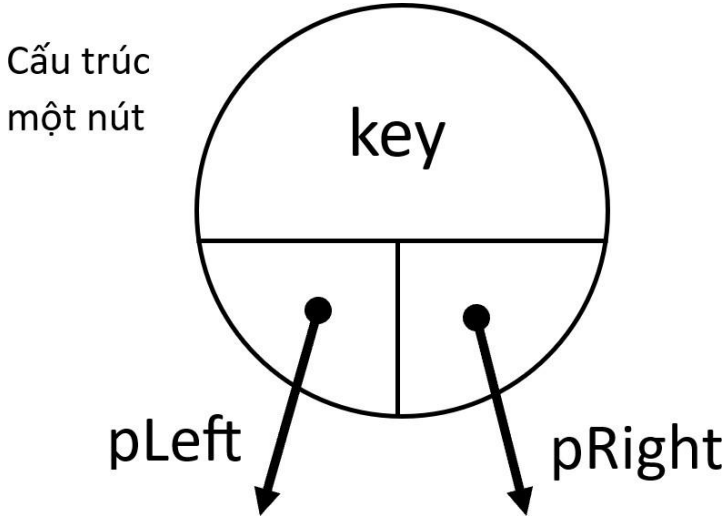
Điều kiện:

- **Mọi nút đều** có giá trị **nhỏ hơn hoặc bằng** giá trị của **các con**.
- **Gốc của cây** sẽ chứa giá trị **nhỏ nhất**.

HIỆN THỰC CÂY NHỊ PHÂN TỔNG QUÁT

MÔ TẢ CẤU TRÚC DỮ LIỆU

- **Trường key:** chứa dữ liệu của node.
- **Trường pLeft:** là con trỏ chỉ đến nút con bên trái.
- **Trường pRight:** là con trỏ chỉ đến nút con bên phải.




```
typedef struct node {
    int key;
    struct node *pLeft;
    struct node *pRight;
} NODE;

typedef NODE* TREE;
```

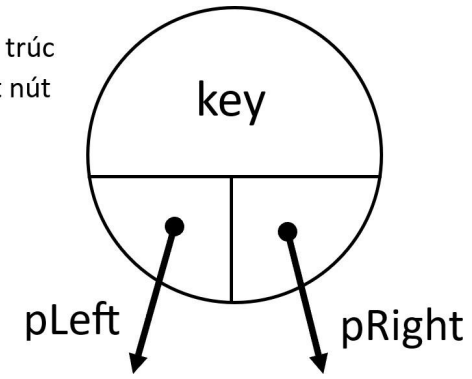
HIỆN THỰC CÂY NHỊ PHÂN TỔNG QUÁT

TÁC VỤ TẠO MỘT NÚT

- Tạo một nút mới với giá trị khóa là key

```
NODE* CreateNode(int key) {
    NODE* newNode = (NODE*)malloc(sizeof(NODE));
    newNode->key = key;
    newNode->pLeft = NULL;
    newNode->pRight = NULL;
    return newNode;
}
```

Cấu trúc một nút



TÁC VỤ KHỞI ĐỘNG CÂY

- Khởi động cây bằng cách đặt gốc bằng NULL

```
void Initialize(TREE &root) {
    root = NULL;
}
```

HIỆN THỰC CÂY NHỊ PHÂN TỔNG QUÁT

TÁC VỤ CHÈN NODE VÀO BÊN TRÁI NHẤT

- Chèn một nút mới vào bên trái nhất của cây con trái

```
void InsertLeft(NODE* parent, int key) {
    if (parent->pLeft == NULL) {
        parent->pLeft = CreateNode(key);
    } else {
        InsertLeft(parent->pLeft, key);
    }
}
```

TÁC VỤ CHÈN NODE VÀO BÊN PHẢI NHẤT

- Chèn một nút mới vào bên phải nhất của cây con phải

```
void InsertRight(NODE* parent, int key) {
    if (parent->pRight == NULL) {
        parent->pRight = CreateNode(key);
    } else {
        InsertRight(parent->pRight, key);
    }
}
```

HIỆN THỰC CÂY NHỊ PHÂN TỔNG QUÁT

TÁC VỤ XÓA NODE BÊN TRÁI NHẤT

- Xóa một nút bên trái nhất của cây con trái

```
void DeleteLeft(NODE* parent) {
    if (parent->pLeft != NULL) {
        if (parent->pLeft->pLeft == NULL
            && parent->pLeft->pRight == NULL) {
            free(parent->pLeft);
            parent->pLeft = NULL;
        } else {
            DeleteLeft(parent->pLeft);
        }
    }
}
```

TÁC VỤ XÓA NODE BÊN PHẢI NHẤT

- Xóa một nút bên phải nhất của cây con phải

```
void DeleteRight(NODE* parent) {
    if (parent->pRight != NULL) {
        if (parent->pRight->pLeft == NULL
            && parent->pRight->pRight == NULL) {
            free(parent->pRight);
            parent->pRight = NULL;
        } else {
            DeleteRight(parent->pRight);
        }
    }
}
```

HIỆN THỰC CÂY NHỊ PHÂN TỔNG QUÁT

TÁC VỤ TÌM KIẾM MỘT NÚT

- Tìm kiếm một nút với giá trị khóa là x

```
NODE* Search(NODE* root, int x) {
    if (root->key == x) {
        return root;
    }
    NODE* result = NULL;
    if (root->pLeft != NULL) {
        result = Search(root->pLeft, x);
    }
    if (result == NULL && root->pRight != NULL) {
        result = Search(root->pRight, x);
    }
    return result;
}
```

TÁC VỤ HỦY TOÀN BỘ CÂY

- Quá trình xóa nút và giải phóng bộ nhớ của nút được thực hiện từ các nút lá đi ngược dần lên

```
void DeleteTree(NODE* root) {
    if (root->pLeft != NULL) {
        DeleteTree(root->pLeft);
        free(root->pLeft);
        root->pLeft = NULL;
    }
    if (root->pRight != NULL) {
        DeleteTree(root->pRight);
        free(root->pRight);
        root->pRight = NULL;
    }
}
```

TÁC VỤ DUYỆT CÂY NLR (NODE-LEFT-RIGHT)

QUY TRÌNH DUYỆT NLR (PREORDER TRAVERSAL)

- **Bước 1 - Bắt đầu từ nút gốc:**

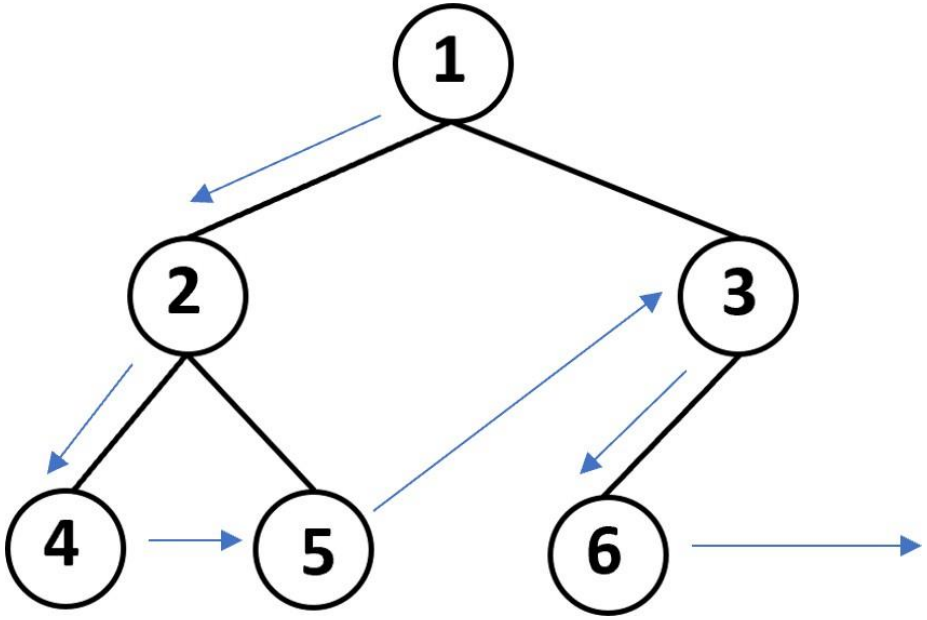
  Thăm nút **gốc** và xử lý nó (ví dụ: in giá trị của nó).

- **Bước 2 - Duyệt cây con trái:**

  Thực hiện đệ quy duyệt NLR trên **cây con trái** (nếu tồn tại) bằng cách quay lại bước 1 với cây con trái làm nút gốc tạm thời.

- **Bước 3 - Duyệt cây con phải:**

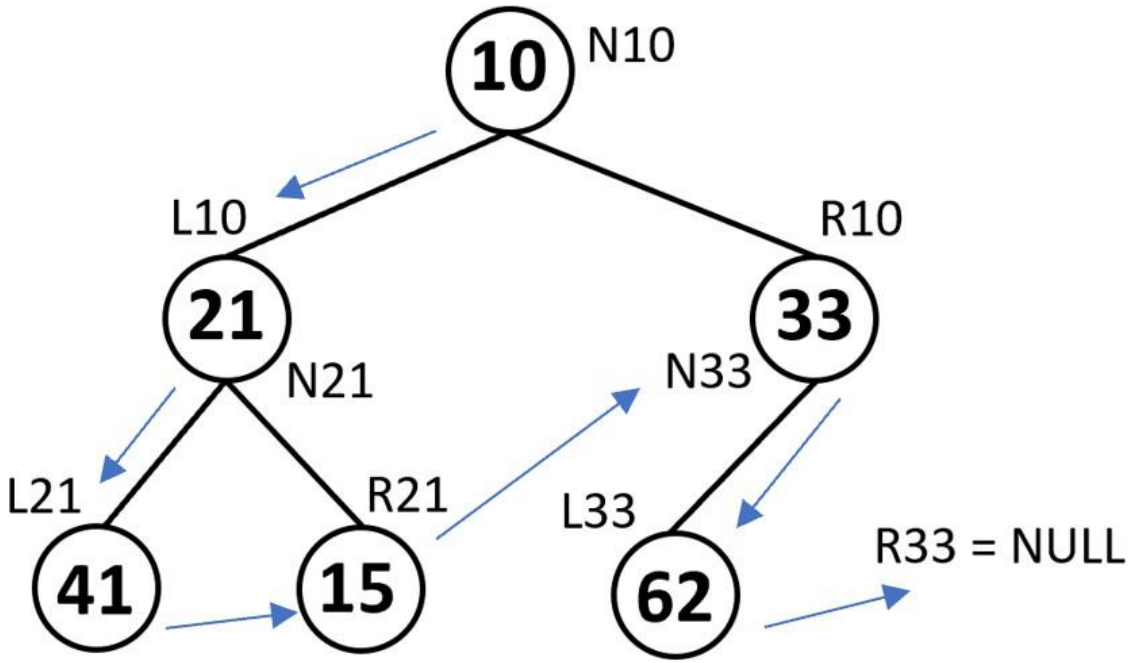
  Thực hiện đệ quy duyệt NLR trên **cây con phải** (nếu tồn tại) bằng cách quay lại bước 1 với cây con phải làm nút gốc tạm thời.



1) N: Thăm nút gốc.
2) L: Duyệt cây con trái.
3) R: Duyệt cây con phải.

TÁC VỤ DUYỆT CÂY NLR (NODE-LEFT-RIGHT)

VÍ DỤ MINH HỌA



- Thăm nút gốc (N10): in giá trị 10.
- Duyệt cây con trái của nút 10 là L10: in giá trị 21.
- Duyệt cây con trái của nút 21 là L21: in ra giá trị 41
- Duyệt cây con phải của nút 21 là R21: in ra giá trị 15
- Duyệt cây con phải của nút 10 là R10: in ra giá trị 33
- Duyệt cây con trái của nút 33 là L33: in ra giá trị 62
- Duyệt cây con phải của nút 33 là R33: không có gì để in ra

- Kết quả NLR: 10, 21, 41, 15, 33, 62.

TÁC VỤ DUYỆT CÂY NLR (NODE-LEFT-RIGHT)

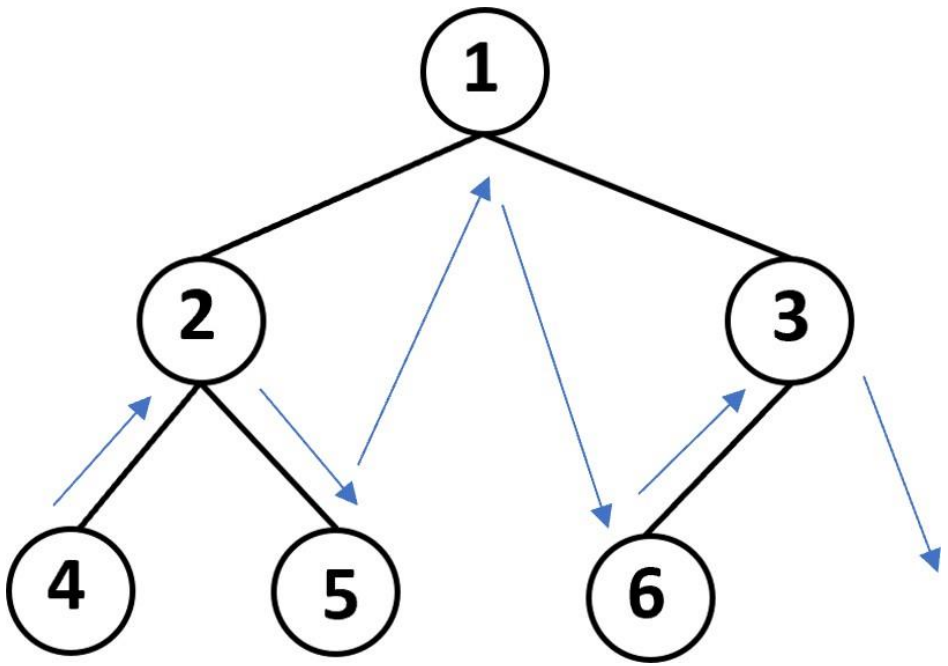
VIẾT HÀM HIỆN THỰC DUYỆT NLR

```
void TraverseNLR(NODE* root) {
    if(root!=NULL)
    {
        printf("%d ", root->key);
        TraverseNLR(root->pLeft);
        TraverseNLR(root->pRight);
    }
}
```

TÁC VỤ DUYỆT CÂY LNR (LEFT-NODE-RIGHT)

QUY TRÌNH DUYỆT LNR (INORDER TRAVERSAL)

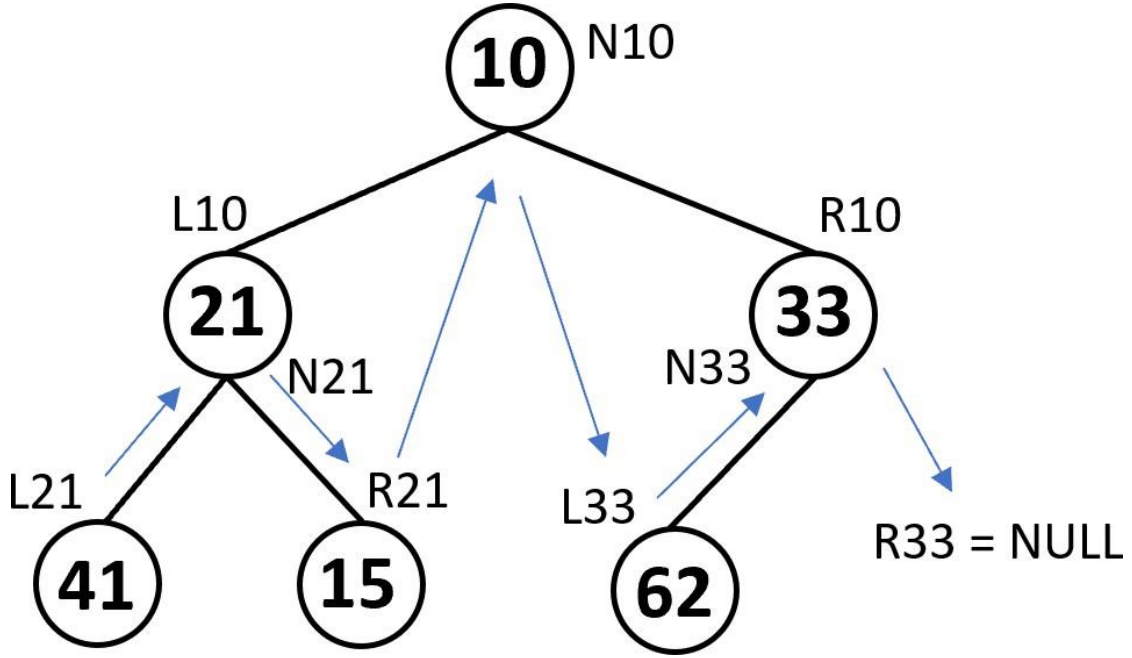
- **Bước 1 - Duyệt cây con trái:**
  Thực hiện duyệt LNR trên cây con trái (nếu nó tồn tại) bằng cách lấy nút gốc của cây con trái làm nút gốc tạm thời.
- **Bước 2 - Thăm nút gốc:**
  Sau khi duyệt xong cây con trái, thăm nút gốc và xử lý nó (ví dụ: in giá trị của nó ra).
- **Bước 3 - Duyệt cây con phải:**
  Thực hiện duyệt LNR trên cây con phải (nếu nó tồn tại) bằng cách lấy nút gốc của cây con phải làm nút gốc tạm thời.



1) L: Duyệt cây con trái.
2) N: Thăm nút gốc.
3) R: Duyệt cây con phải.

TÁC VỤ DUYỆT CÂY LNR (LEFT-NODE-RIGHT)

VÍ DỤ MINH HỌA



- Duyệt cây con trái (L21) của cây L10 : in giá trị 41.
- Thăm nút N21: in ra giá trị 21.
- Duyệt cây con phải (R21) của cây L10 : in giá trị 15.
- Quay về nút gốc N10: in ra giá trị 10
- Duyệt cây con trái (L33) của cây R10 : in giá trị 62.
- Thăm nút N33: in ra giá trị 33.
- Duyệt cây con phải (R33) của cây R10 : không có gì để in ra.

- Kết quả LNR: 41, 21, 15, 10, 62, 33

TÁC VỤ DUYỆT CÂY LNR (LEFT-NODE-RIGHT)

VIẾT HÀM HIỆN THỰC DUYỆT LNR

```
void TraverseLNR(NODE* root) {
    if(root!=NULL)
    {
        TraverseLNR(root->pLeft);
        printf("%d ", root->key);
        TraverseLNR(root->pRight);
    }
}
```

TÁC VỤ DUYỆT CÂY LRN (LEFT-RIGHT-NODE)

QUY TRÌNH DUYỆT LRN (POSTORDER TRAVERSAL)

- **Bước 1 - Duyệt cây con trái:**

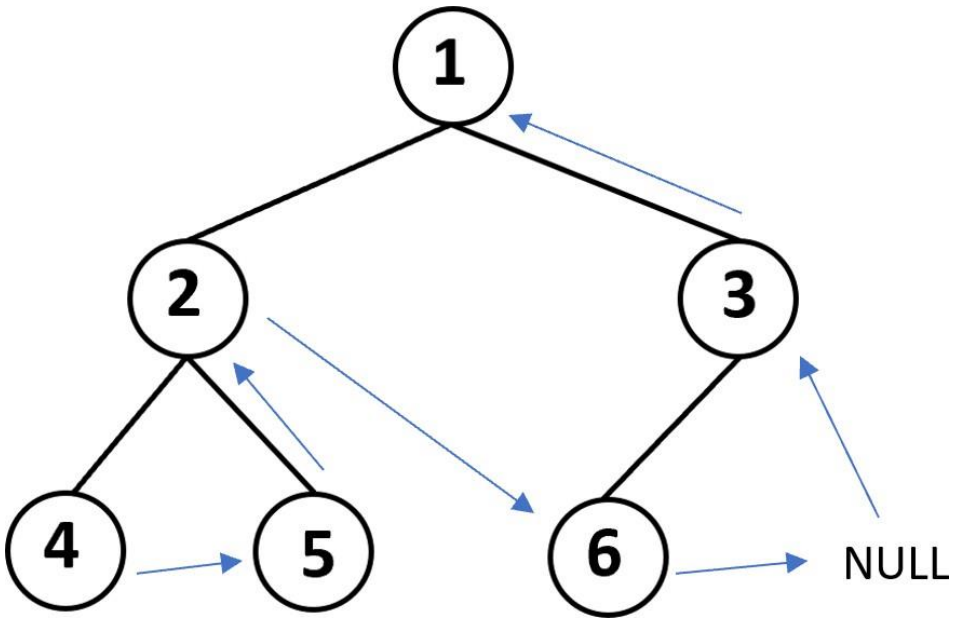
  Thực hiện duyệt LRN trên (nếu nó tồn tại) bằng cách lấy nút gốc **cây con trái** của cây con trái làm nút gốc tạm thời.

- **Bước 2 - Duyệt cây con phải:**

  Thực hiện duyệt LRN trên **cây con phải** (nếu nó tồn tại) bằng cách lấy nút gốc của cây con phải làm nút gốc tạm thời.

- **Bước 3 - Thăm nút gốc:**

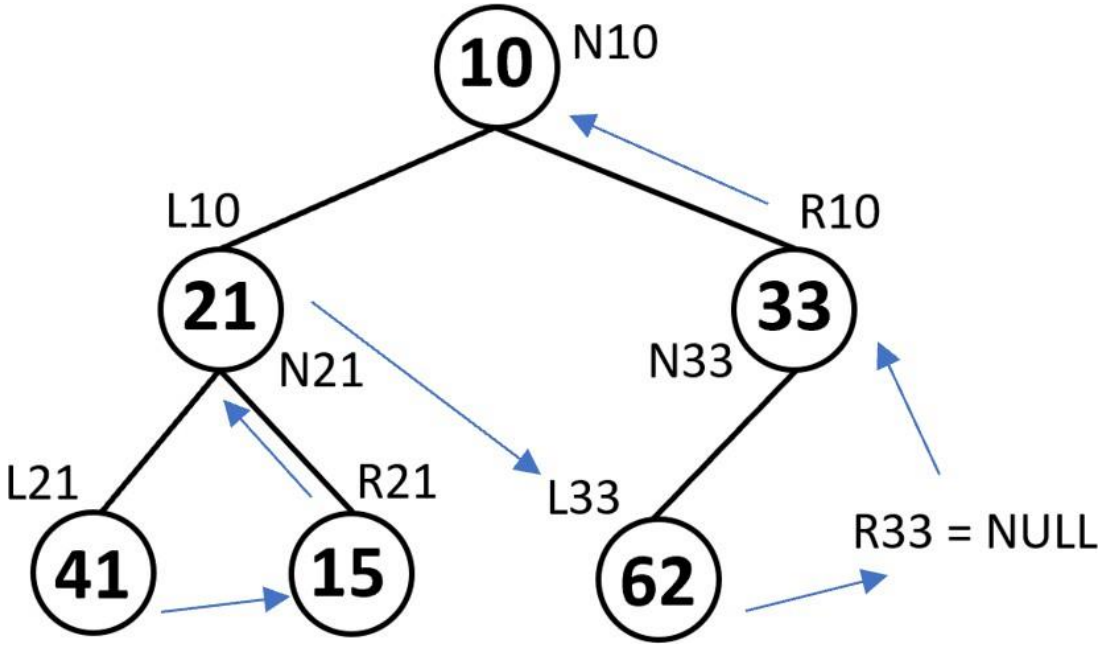
  Sau khi duyệt xong cây con trái và cây con phải, **thăm nút gốc** và xử lý nó (ví dụ: in giá trị của nó ra).



1) L: Duyệt cây con trái.
2) R: Duyệt cây con phải.
3) N: Thăm nút gốc.

TÁC VỤ DUYỆT CÂY LRN (LEFT-RIGHT-NODE)

VÍ DỤ MINH HỌA



- Duyệt cây con trái (L21) của cây L10 : in giá trị 41.
- Duyệt cây con phải (R21) của cây L10 : in giá trị 15.
- Thăm nút N21: in ra giá trị 21.
- Duyệt cây con trái (L33) của cây R10 : in giá trị 62.
- Duyệt cây con phải (R33) của cây R10 : không có gì để in.
- Thăm nút N33: in ra giá trị 33.
- Quay về nút gốc N10: in ra giá trị 10

- Kết quả LRN: 41, 15, 21, 62, 33, 10

TÁC VỤ DUYỆT CÂY LRN (LEFT-RIGHT-NODE)

VIẾT HÀM HIỆN THỰC DUYỆT LRN

```
void TraverseLRN(NODE* root) {
    if(root!=NULL)
    {
        TraverseLRN(root->pLeft);
        TraverseLRN(root->pRight);
        printf("%d ", root->key);
    }
}
```

BÀI TẬP - cây nhị phân tổng quát

Dùng cách vẽ bằng tay, hãy tạo **cây nhị phân tổng quát** theo thứ tự nhập sau đây:

27 19 10 21 3 15 41 50 30 27

a) Vẽ cây nhị phân tổng quát từ dãy số trên (với thao tác lần lượt chèn vào cây bên trái, chèn vào cây bên phải).

b) Lần lượt duyệt cây theo ba phương pháp PreOrder, InOrder, PostOrder.

c) Đếm số lượng (bằng cách khoanh tròn trên hình vẽ) các nút lá, các nút có hai con, các nút có một con trái, các nút có một con phải, các nút có một con (hoặc con trái hoặc con phải đều được).

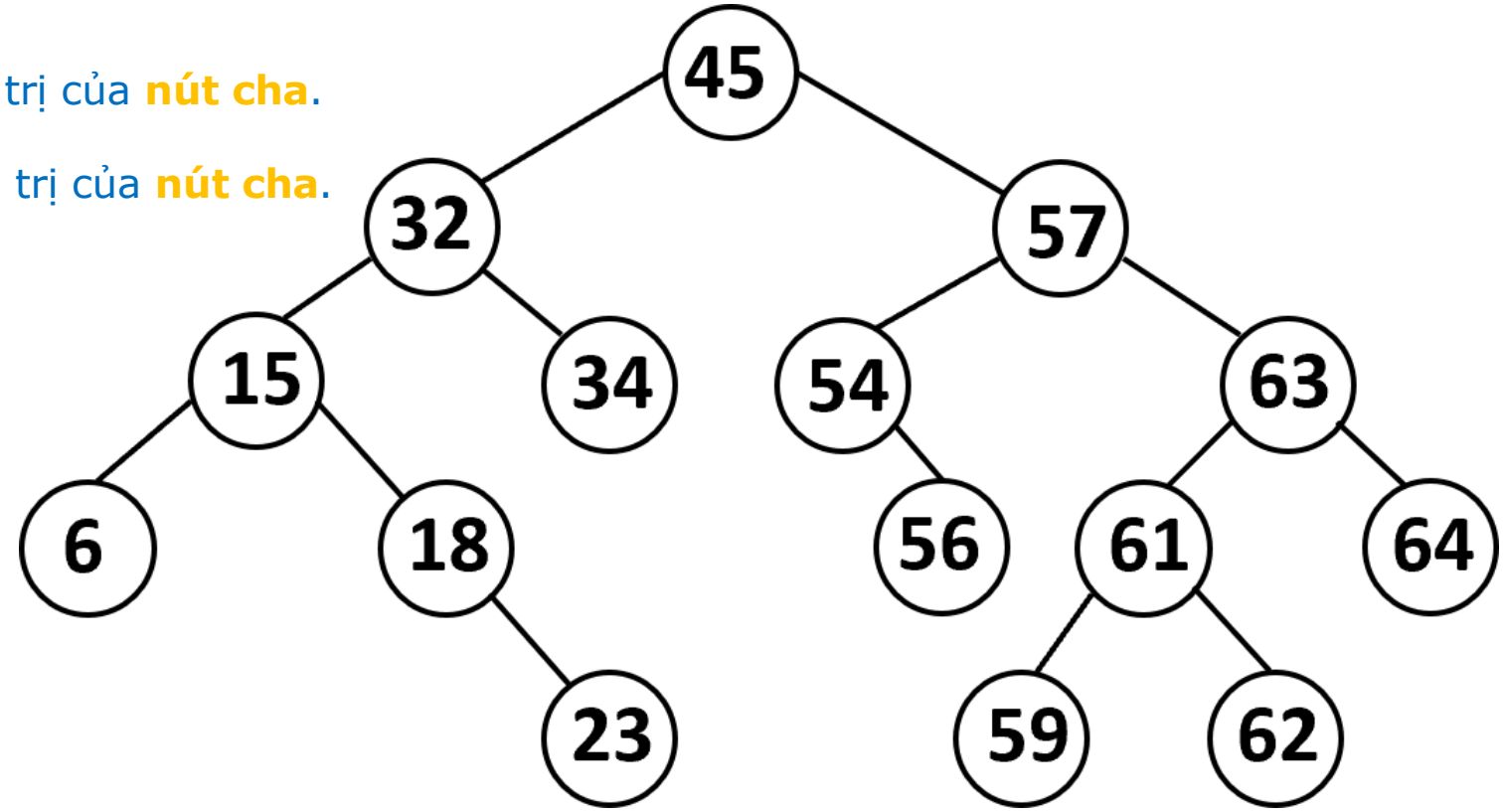
CÂY VÀ CÂY NHỊ PHÂN TÌM KIẾM

6.3 - CÂY NHỊ PHÂN TÌM KIẾM (BST)

KHÁI NIỆM CÂY NHỊ PHÂN TÌM KIẾM

Cây nhị phân tìm kiếm (**Binary Search Tree - BST**):

- Mỗi nút có tối đa hai con
- Giá trị của nút **con trái nhỏ** hơn giá trị của **nút cha**.
- Giá trị của nút **con phải lớn hơn** giá trị của **nút cha**.



HIỆN THỰC CÂY NHỊ PHÂN TÌM KIẾM - BST

XÂY DỰNG CẤU TRÚC DỮ LIỆU CHO CÂY BST

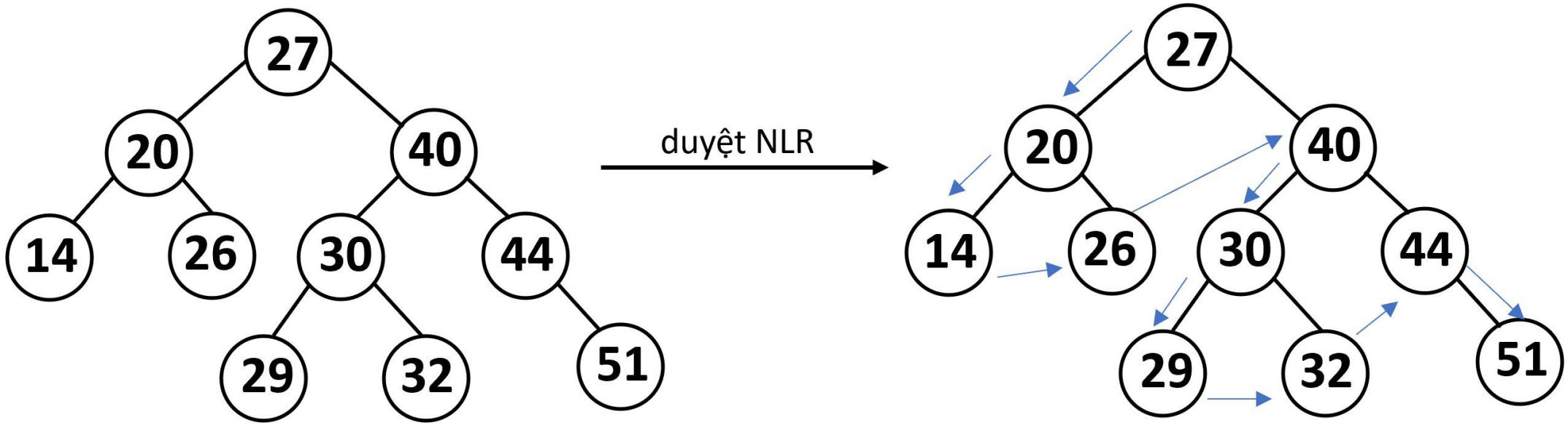
- Khai báo cấu trúc dữ liệu cho một nút của cây và cây.

```
typedef struct tnode
{
    int key;
    struct tnode* pLeft;
    struct tnode* pRight;
}TNODE;

typedef TNODE* TREE;
```

BA TÁC VỤ DUYỆT CÂY BST

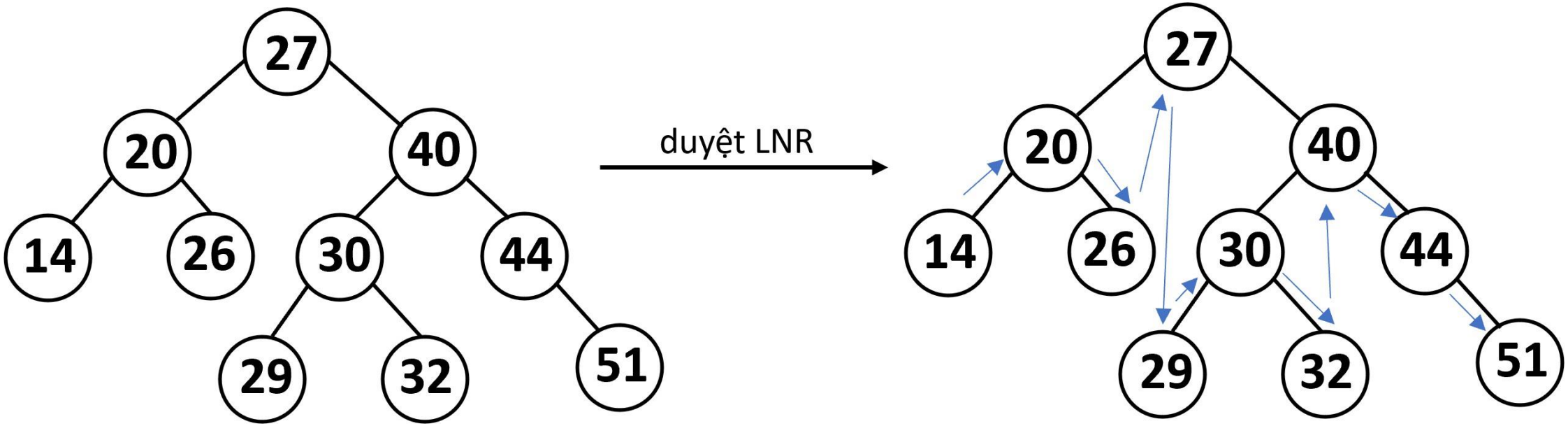
PHƯƠNG PHÁP DUYỆT NLR (PreOrder Traversal)



- Kết quả NLR: **27, 20, 14, 26, 40, 30, 29, 32, 44, 51**

BA TÁC VỤ DUYỆT CÂY BST

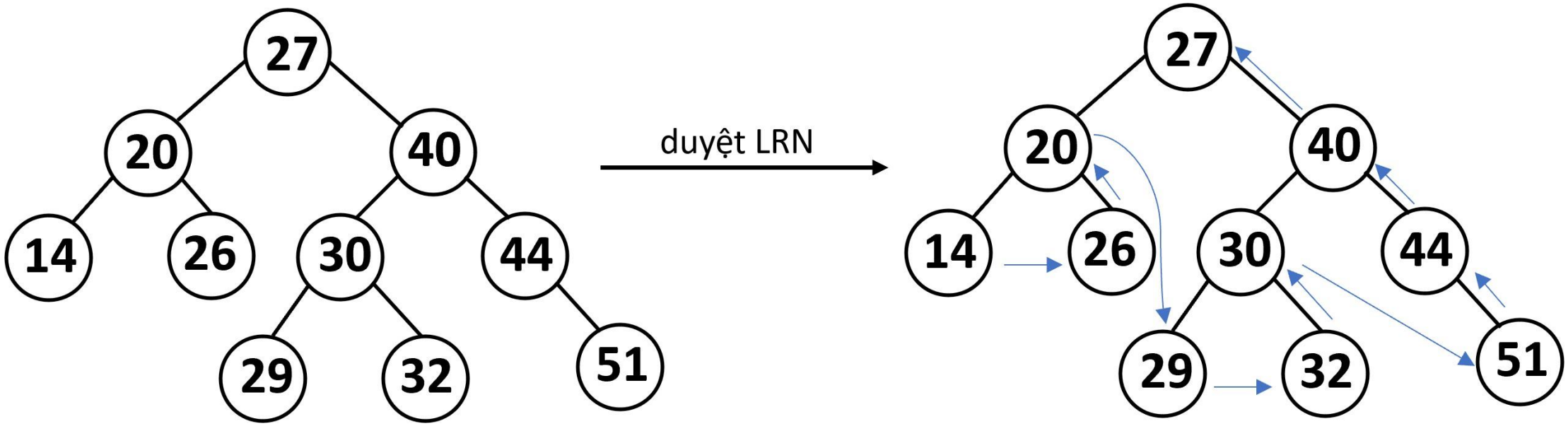
PHƯƠNG PHÁP DUYỆT LNR (InOrder Traversal)



- Kết quả LNR: **14, 20, 26, 27, 29, 30, 32, 40, 44, 51**

BA TÁC VỤ DUYỆT CÂY BST

PHƯƠNG PHÁP DUYỆT LRN (PostOrder Traversal)

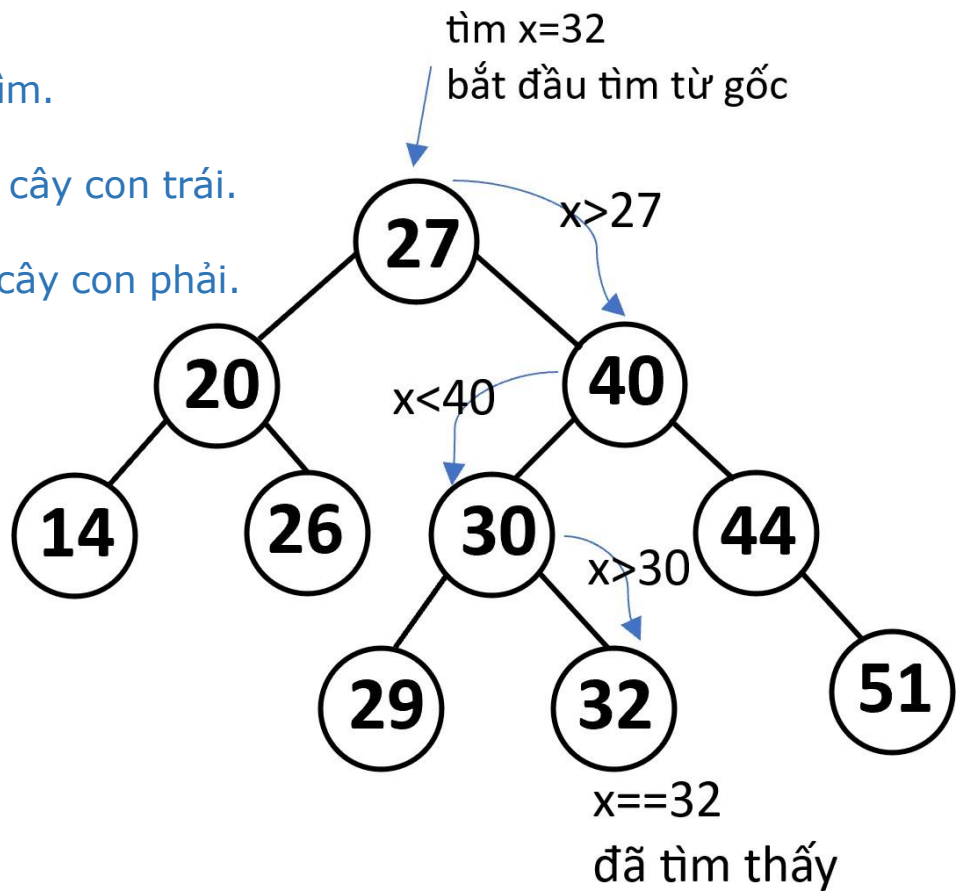


- Kết quả LRN: **14, 26, 20, 29, 32, 30, 51, 44, 40, 27**

TÁC VỤ TÌM KIẾM

CÁC BƯỚC THỰC HIỆN

- Bắt đầu từ nút gốc của cây BST.
- So sánh giá trị cần tìm với giá trị của nút hiện tại:
  - Nếu giá trị x bằng giá trị của nút hiện tại, ta đã tìm thấy giá trị cần tìm.
  - Nếu giá trị x nhỏ hơn giá trị của nút hiện tại, tiếp tục tìm kiếm trong cây con trái.
  - Nếu giá trị x lớn hơn giá trị của nút hiện tại, tiếp tục tìm kiếm trong cây con phải.
- Di chuyển:
  - Di chuyển xuống cây con tương ứng
  - Lặp lại quá trình so sánh và di chuyển cho đến khi tìm thấy giá trị x
  - Hoặc gặp nút lá (trường hợp giá trị x không có trong cây).
- Kết thúc tìm kiếm:
  - Nếu tìm thấy giá trị x, trả về vị trí của nút chứa giá trị x.
  - Nếu không tìm thấy giá trị x trong cây, thông báo rằng giá trị x không tồn tại trong cây.



TÁC VỤ TÌM KIẾM

VIẾT HÀM TÌM KIẾM

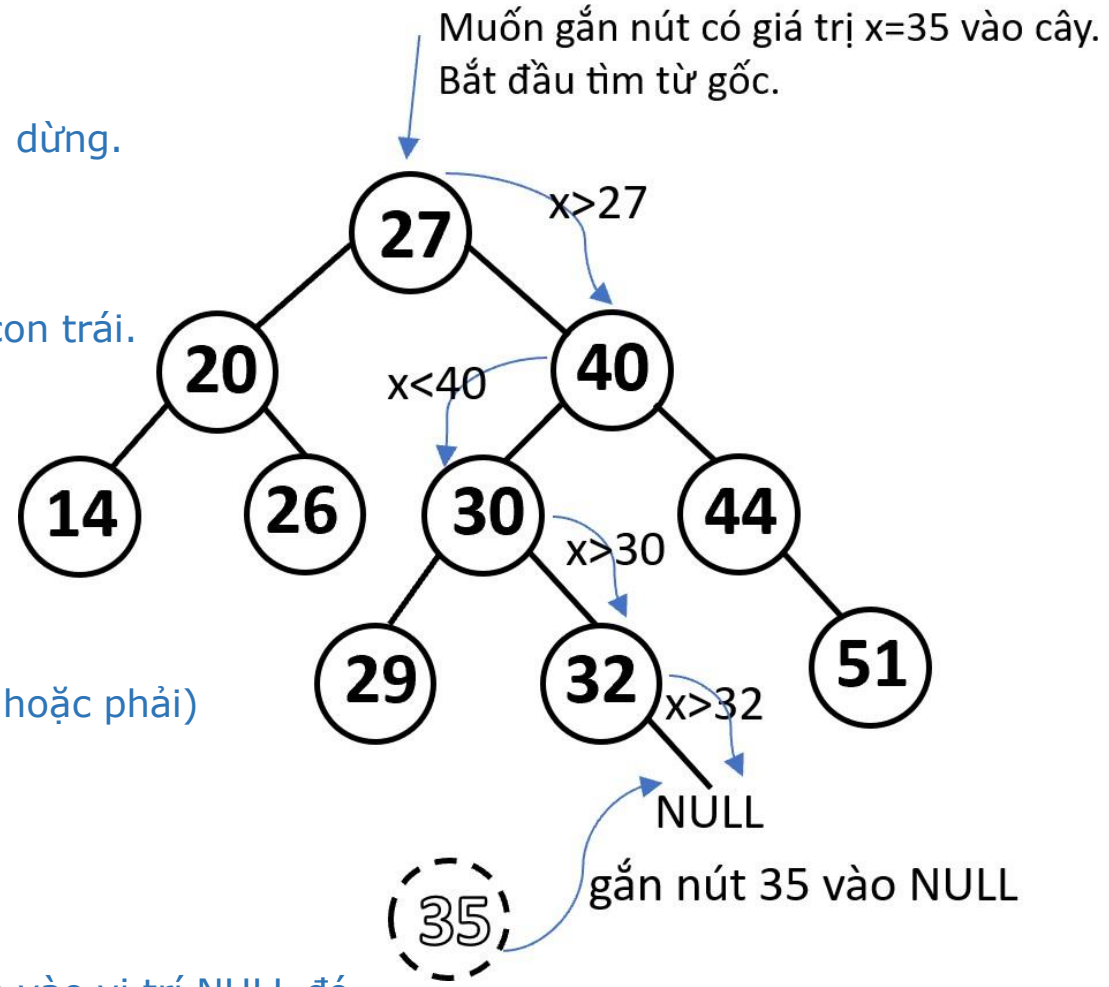
- Hàm trả về nút tìm thấy, nếu không tìm thấy thì hàm trả về NULL

```
TNODE* SearchBST(TREE t, int x){
	if(t!=NULL){
		if(t->key == x) return t;
		else{
			if(x<t->key) return SearchBST(t->pLeft,x);
			else return SearchBST(t->pRight,x);
		}
	}
	else  return NULL;
}
```

TÁC VỤ THÊM MỘT NÚT

CÁC BƯỚC THỰC HIỆN

- **Bước 1:** Khởi động từ nút gốc của cây
  - Bắt đầu từ nút gốc của cây BST.
  - Nếu cây rỗng (gốc cây là NULL), tạo nút mới, gán làm gốc rồi dừng.
- **Bước 2:** So sánh giá trị cần thêm với giá trị của nút hiện tại
  - Nếu giá trị cần thêm nhỏ hơn giá trị nút hiện tại, đi tiếp cây con trái.
  - Nếu lớn hơn, đi tiếp cây con phải.
  - Nếu bằng, không thêm gì cả vì BST không chấp nhận trùng.
- **Bước 3:** Di chuyển đến vị trí nút con tương ứng
  - Tiếp tục so sánh và di chuyển xuống cây con tương ứng (trái hoặc phải) cho đến khi gặp một vị trí NULL.
- **Bước 4:** Thêm nút mới
  - Khi gặp vị trí NULL, tạo một nút mới với giá trị cần thêm, gán vào vị trí NULL đó.



TÁC VỤ THÊM MỘT NÚT

VIẾT HÀM THÊM MỘT NÚT

- Nếu thêm nút thành công thì hàm trả về 1, ngược lại thì hàm trả về 0 hoặc -1.

```
int AddNode(TREE &t, int x)
{
	if(t!=NULL){
		if(x==t->key) return 0; //trùng
		else {
			if(x<t->key) return AddNode(t->pLeft,x);
			else return AddNode(t->pRight,x);
		}
	}
	else{
		t= new TNODE;
		if(t==NULL) return -1; //không cấp phát được
		t->key = x;
		t->pLeft = t->pRight = NULL;
		return 1;
	}
}
```

TÁC VỤ XÓA MỘT PHẦN TỬ

Áp dụng giải thuật tìm kiếm để tìm thấy nút cần xóa.

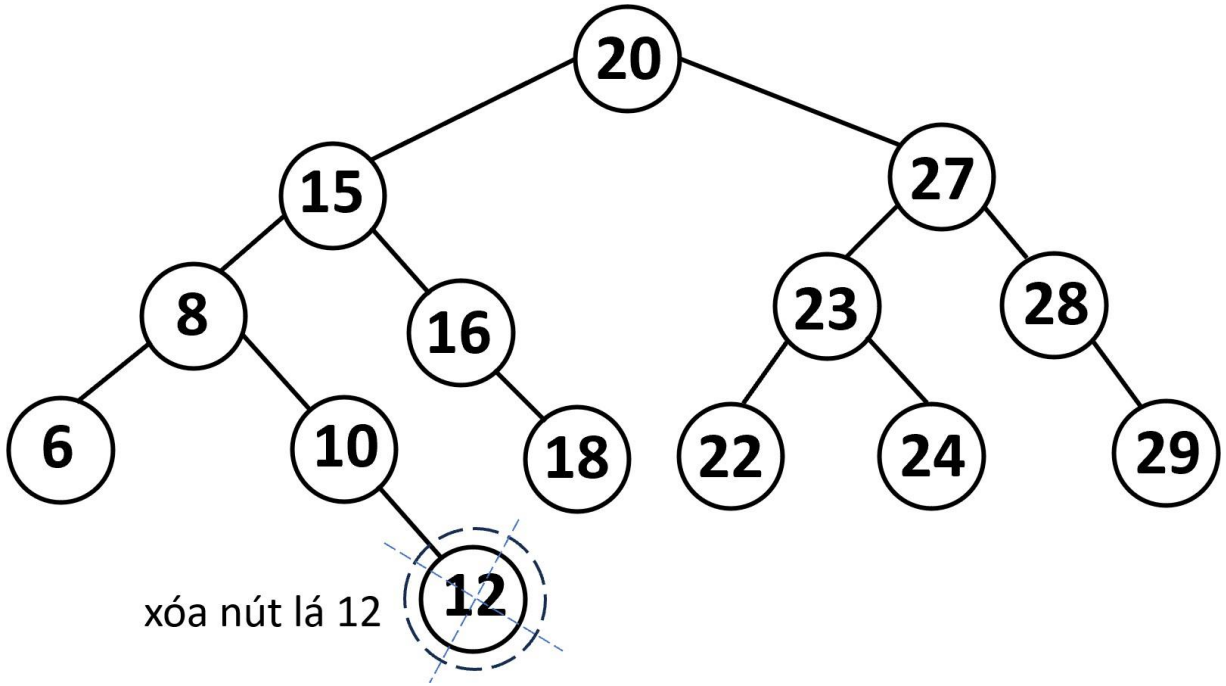
Sau đó tiến hành xóa nút.

Có 3 trường hợp xảy ra tại nút cần xóa:

- **Trường hợp 1:** Nút cần xóa là nút lá (không có con)
- **Trường hợp 2:** Nút cần xóa có một con
- **Trường hợp 3:** Nút cần xóa có hai con

TRƯỜNG HỢP 1 - NÚT CẦN XÓA LÀ NÚT LÁ

- Chỉ cần xóa nút đó khỏi cây.
- Cập nhật con trỏ của nút cha để trỏ đến NULL.



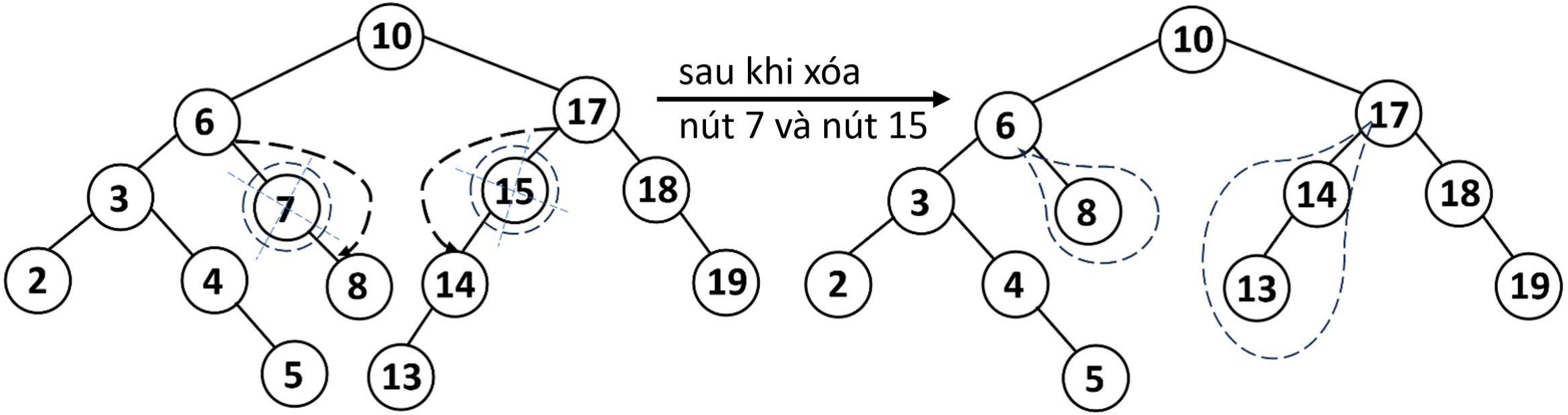
Minh họa xóa một nút lá 12

TRƯỜNG HỢP 2 - NÚT CẦN XÓA CÓ MỘT CON

- Xác định xem con của nút cần xóa nằm bên trái hay bên phải.
- Kết nối nút cha của nút cần xóa với con của nút cần xóa.
- Xóa nút cần xóa.

Minh họa xóa nút có một con như sau.

- Nút 7 có một con bên phải.
- Nút 15 có một con bên trái.
- Tiến hành xóa nút 7 và xóa nút 15.



TRƯỜNG HỢP 3 - NÚT CẦN XÓA CÓ HAI CON

TÌM NÚT THẾ MẠNG

Để tìm nút thế mạng ta thực hiện như sau:

- Đứng tại nút đang sắp bị xóa, ta đi tìm nút thế mạng cho nó. Có hai cách tìm nút thế mạng, đó là:
  - **Cách 1: Tìm nút trái nhất của cây con bên phải.**

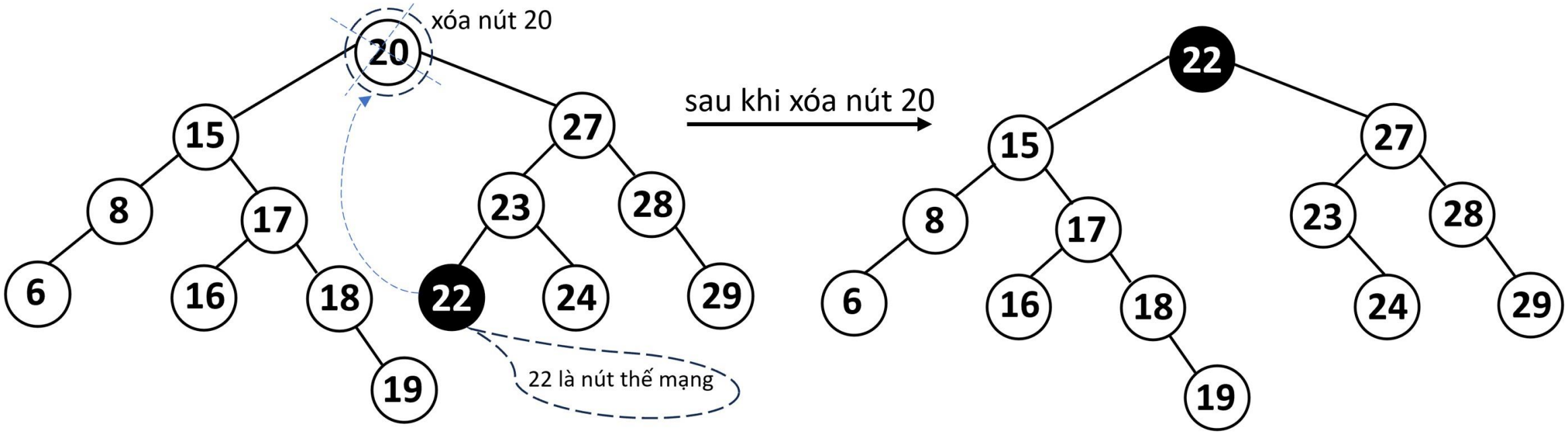
    (Tức là nút nhỏ nhất của cây con bên phải)
  - **Cách 2: Tìm nút phải nhất của cây con bên trái.**

    (Tức là nút lớn nhất của cây con bên trái)
- Sao chép giá trị của nút thế mạng vào nút cần xóa.
- Xóa nút thế mạng bằng cách gọi đệ quy quá trình xóa.

TRƯỜNG HỢP 3 - NÚT CẦN XÓA CÓ HAI CON

ÁP DỤNG CÁCH 1

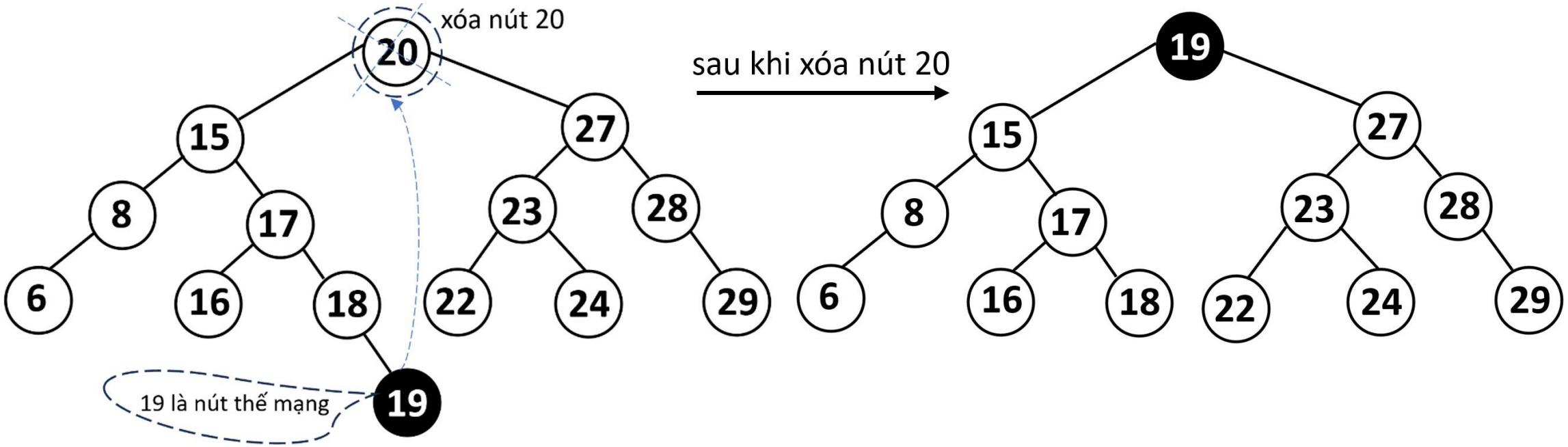
- Tìm nút trái nhất của cây con bên phải của nút 20.



TRƯỜNG HỢP 3 - NÚT CẦN XÓA CÓ HAI CON

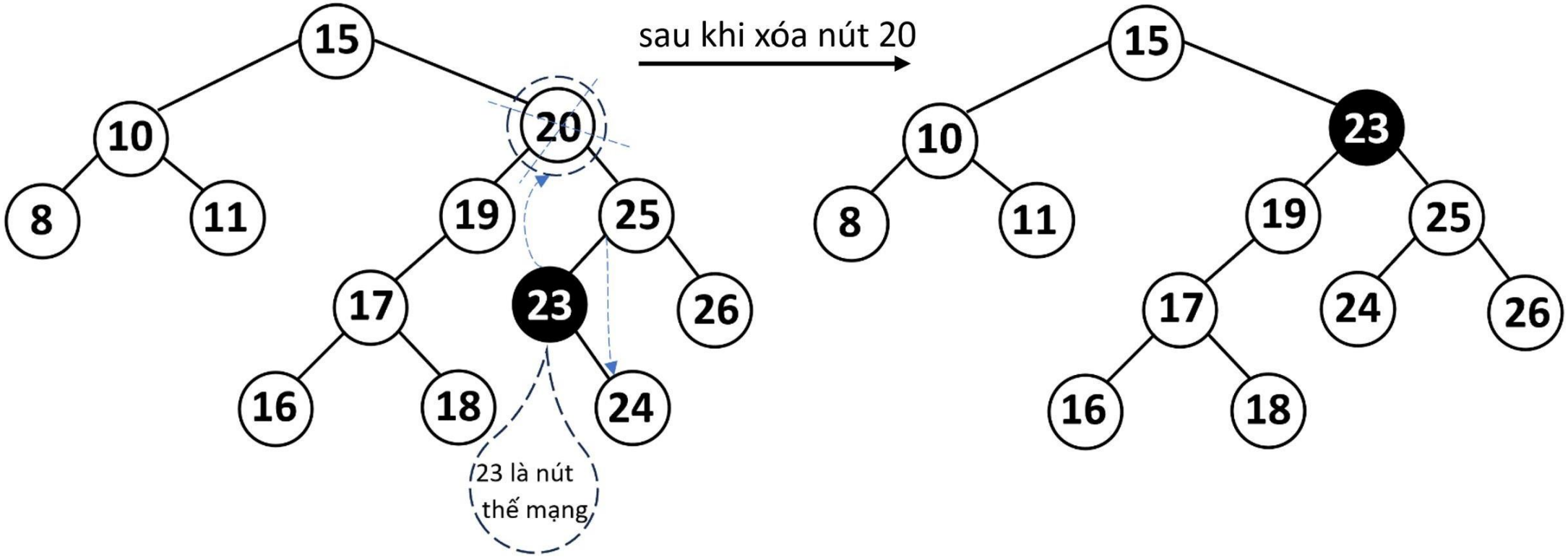
ÁP DỤNG CÁCH 2

- Tìm nút phải nhất của cây con bên trái của nút 20.



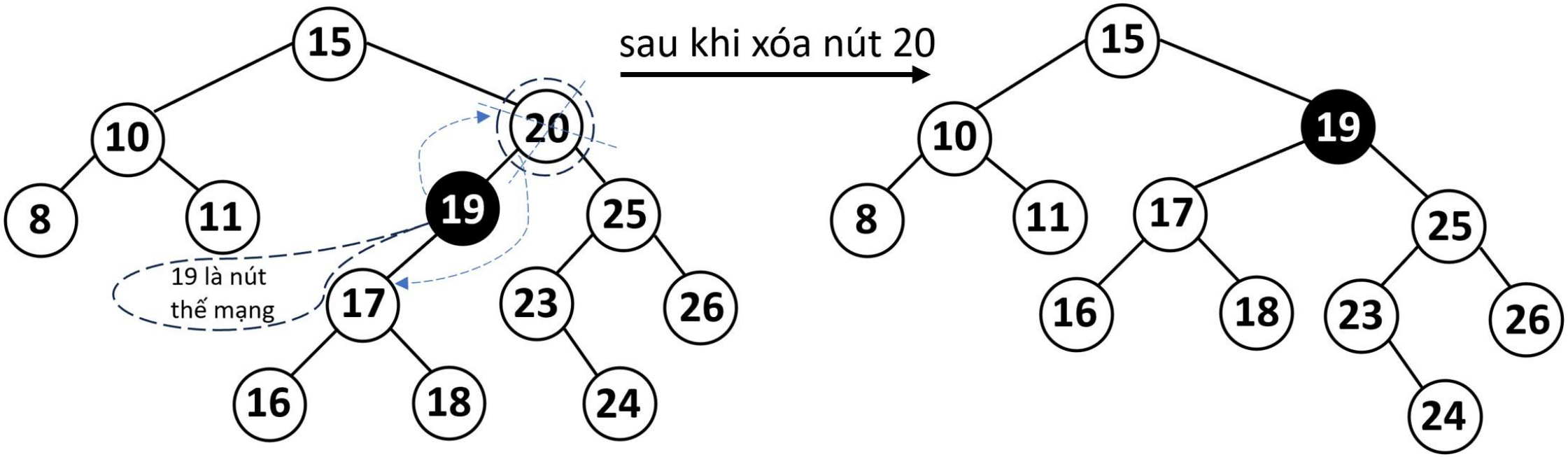
TRƯỜNG HỢP 3 - NÚT CẦN XÓA CÓ HAI CON

THÊM MỘT VÍ DỤ (ÁP DỤNG CÁCH 1)



TRƯỜNG HỢP 3 - NÚT CẦN XÓA CÓ HAI CON

THÊM MỘT VÍ DỤ (ÁP DỤNG CÁCH 2)

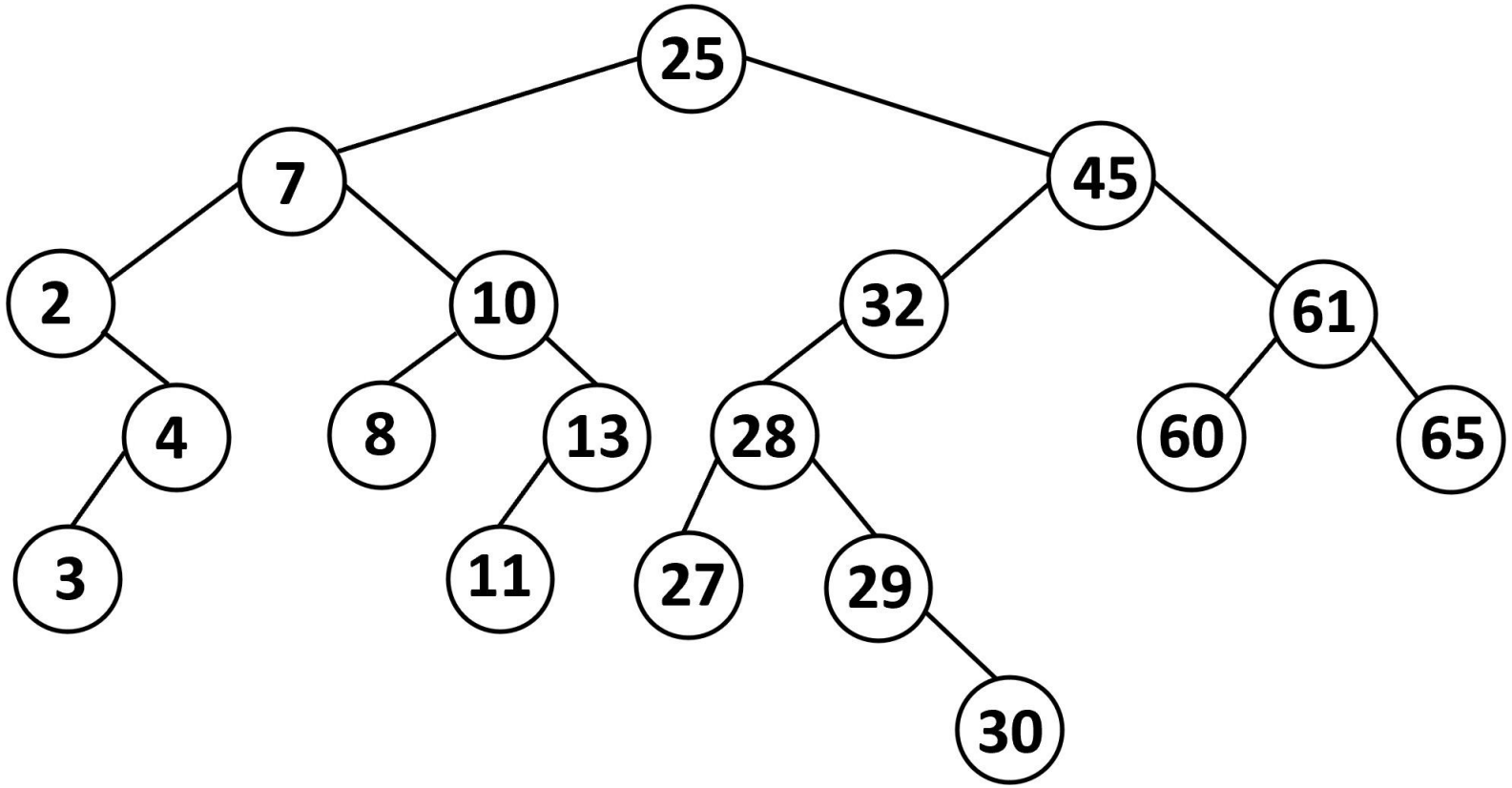


NHẬP NỘI DUNG CHO CÂY

- Hãy tạo cây bằng cách lần lượt lấy các giá trị từ dãy số sau đây chèn vào cây:

{25, 7, 10, 13, 2, 4, 3, 8, 11, 45, 32, 61, 65, 60, 28, 27, 29, 30}

Ta có cây kết quả như sau.



NHẬP NỘI DUNG CHO CÂY

HÀM NHẬP NỘI DUNG CHO CÂY

```
void TreeInput(TREE &t){
	int x; t=NULL;
	while(1){
		printf("Nhap x="); scanf("%d",&x);
		int kq = AddNode(t,x);
		if(kq==0 || kq==-1) break;
	}
}
//hoặc có thể tạo cây từ mảng một chiều có sẵn
void TreeInput(TREE &t) {
	int a[18]={25,7,10,13,2,4,3,8,11,45,32,61,65,60,28,27,29,30};
	int n=18; t=NULL;
	for(int i=0;i<n;i++)
		AddNode(t,a[i]);
}
```

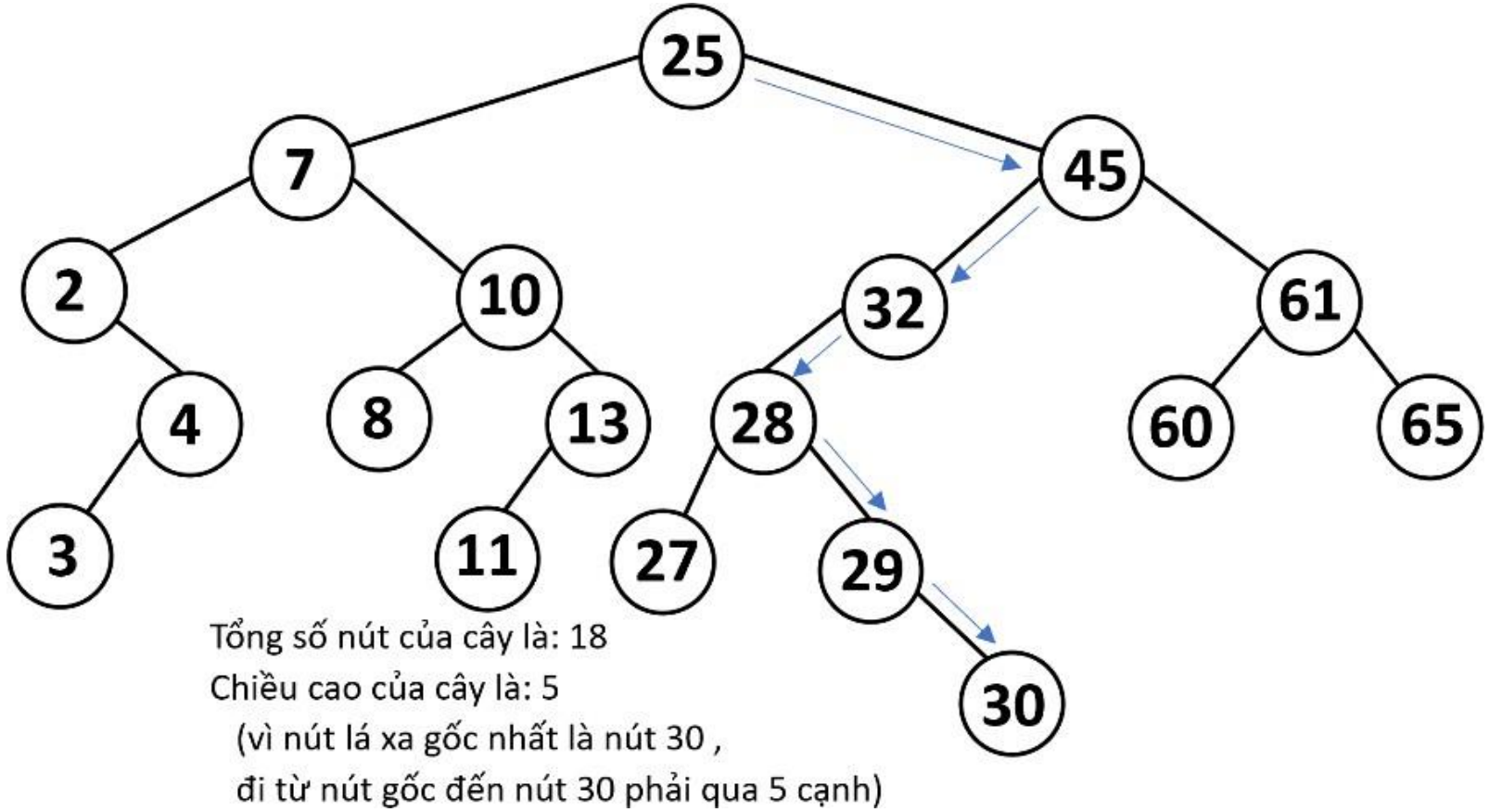
TÁC VỤ HỦY CÂY

- Hàm sau đây hủy toàn bộ các nút của cây

```
void DeleteTree(TREE &t){
	if(t!=NULL){
		DeleteTree(t->pLeft);
		DeleteTree(t->pRight);
		delete t;
	}
}
```

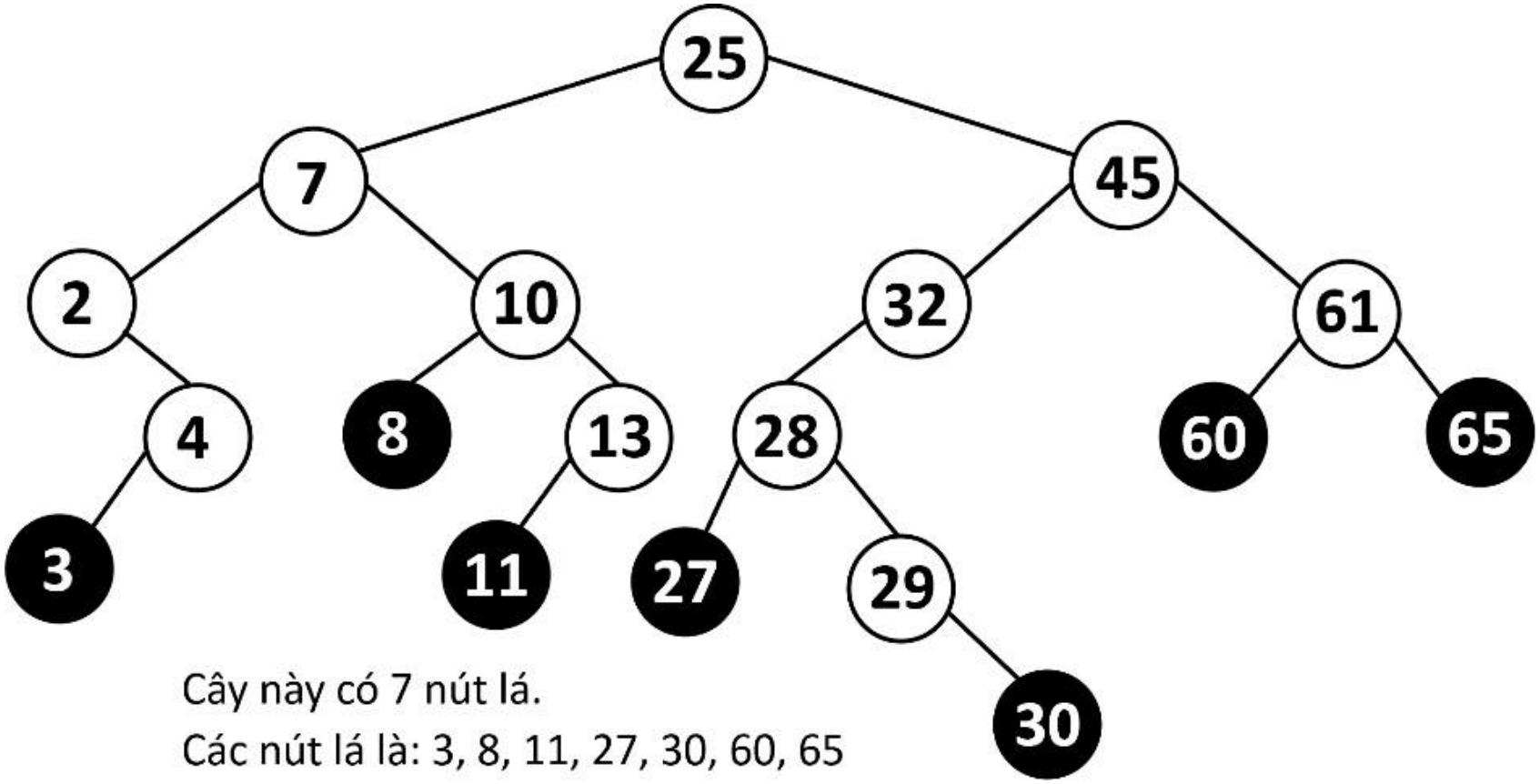
CÁC TÁC VỤ NÂNG CAO

- Đếm số lượng nút của cây (xem code trong giáo trình)



CÁC TÁC VỤ NÂNG CAO

- Đếm số lượng nút lá của cây (xem code trong giáo trình)

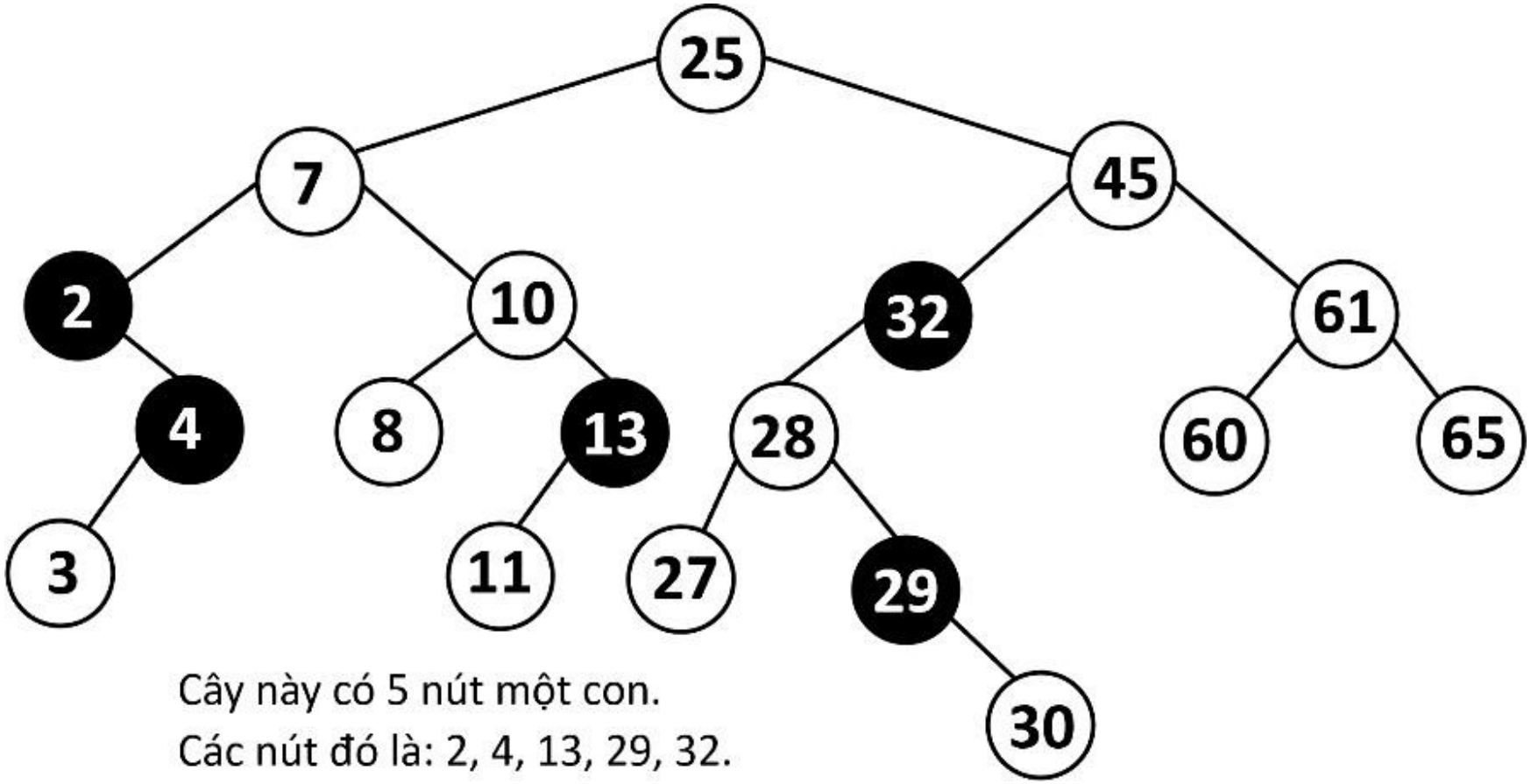


Cây này có 7 nút lá.

Các nút lá là: 3, 8, 11, 27, 30, 60, 65

CÁC TÁC VỤ NÂNG CAO

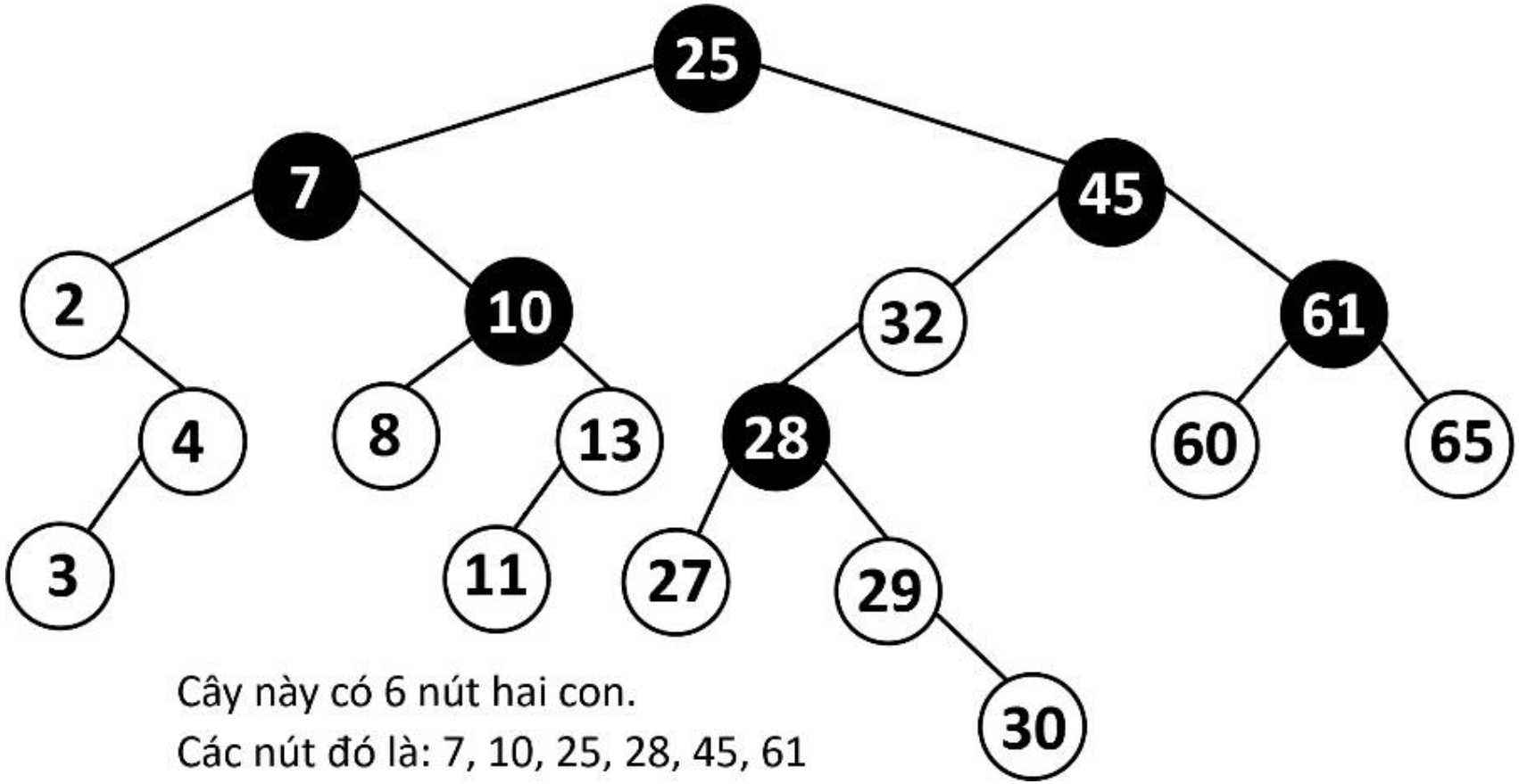
- Đếm số lượng nút có một con (xem code trong giáo trình)



Cây này có 5 nút một con.
Các nút đó là: 2, 4, 13, 29, 32.

CÁC TÁC VỤ NÂNG CAO

- Đếm số lượng nút có hai con (xem code trong giáo trình)



Cây này có 6 nút hai con.

Các nút đó là: 7, 10, 25, 28, 45, 61

CÁC TÁC VỤ NÂNG CAO KHÁC

- Tính tổng giá trị các nút của cây
- Tính tổng các nút có giá trị chẵn
- Tính chiều cao cây
- Tìm khoảng cách từ gốc đến nút có giá trị x
- Xuất ra tất cả các nút tại mức m

Vân… vân…

BÀI TẬP

BÀI 1: Cho trước một mảng a kiểu int có n phần tử. Hãy tạo một cây nhị phân tổng quát có n nút, mỗi nút lưu một phần tử của mảng. Viết code hiện thực các tác vụ sau đây.

a) Viết các hàm duyệt cây theo thứ tự NLR, LNR, LRN và duyệt theo mức (level).

b) Viết hàm tìm nút có giá trị là x.

c) Viết hàm xác định chiều cao của cây

d) Viết hàm đếm số nút trên cây.

e) Viết hàm đếm số lượng nút lá của cây.

f) Viết các hàm đếm số nút thỏa điều kiện: đủ hai cây con, có giá trị nhỏ hơn k, có giá trị lớn hơn giá trị của nút con trái và nhỏ hơn giá trị của nút con phải, có chiều cao cây con trái bằng chiều cao cây con phải.

BÀI TẬP

BÀI 2: Cho trước một mảng a kiểu int có n phần tử. Hãy tạo một cây nhị phân tổng quát có n nút, mỗi nút lưu một phần tử của mảng. Viết code hiện thực các tác vụ sau đây.

a) Viết hàm đếm số nút lá có giá trị lớn hơn x.

b) Viết hàm tìm nút có giá trị lớn nhất và nút có giá trị nhỏ nhất trong cây.

c) Viết hàm kiểm tra cây có cân bằng chưa.

d) Viết hàm kiểm cây cân bằng hoàn toàn chưa.

e) Viết hàm kiểm tra xem tất cả các nút có đủ hai con chưa (trừ các nút lá).

f) Viết hàm kiểm tra xem mọi nút trên cây đều nhỏ hơn x chưa.

BÀI 3: Khai báo cấu trúc dữ liệu và viết lại toàn bộ các tác vụ đã được giải thích trong bài học về cây nhị phân tìm kiếm BST. Viết hàm main gọi chạy các hàm đã viết.

BÀI TẬP

BÀI 4: Dùng cách vẽ bằng tay, hãy tạo cây nhị phân tìm kiếm theo thứ tự nhập sau đây:

27 19 10 21 3 15 41 50 30 27

a) Vẽ cây nhị phân tìm kiếm từ dãy số trên.

b) Lần lượt duyệt cây theo ba phương pháp PreOrder, InOrder, PostOrder.

c) Vẽ lại các quá trình chèn các nút 29, sau đó chèn nút 31, sau đó chèn tiếp nút 49.

d) Với cây kết quả sau khi chèn ba nút nói trên, hãy vẽ lại từng bước quá trình xóa nút 27 (lần lượt giải với hai cách tìm nút thế mạng đã được giải thích trong bài học).

e) Đếm số lượng (bằng cách khoanh tròn trên hình vẽ) các nút lá, các nút có hai con, các nút có một con trái, các nút có một con phải, các nút có một con (hoặc con trái hoặc con phải đều được).

BÀI TẬP

BÀI 5: Cho dãy số theo thứ tự nhập từ trái sang phải như sau:

20 15 35 30 11 13 17 36 47 16 38 28 14

a) Hãy vẽ cây nhị phân tìm kiếm cho dãy số trên.

b) Hãy cho biết kết quả duyệt cây trên theo thứ tự trước, giữa và sau.

c) Cho biết độ cao của cây, các nút lá, các nút có bậc 2.

d) Vẽ lại cây sau khi thêm nút: 25 và 91

e) Trình bày từng bước và vẽ lại cây sau khi lần lượt xoá các nút: 11 và 35

BÀI TẬP

BÀI 6: Hãy tạo cây nhị phân tìm kiếm từ dãy số nguyên được nhập vào theo thứ tự như sau

10 25 55 20 5 3 8 4 21 Hãy cho biết các thông tin của cây:

a) Số nút lá (nút bậc 0)

b) Số nút có 1 cây con (nút bậc 1)

c) Số nút chỉ có 1 cây con phải

d) Số nút có 1 cây con trái

e) Số nút 2 cây con (nút bậc 2)

f) Độ cao của cây

g) Số nút của cây

h) Các nút trên từng mức của cây

i) Độ dài đường đi từ gốc đến nút 21, từ gốc đến nút 3.

Q&A!